Chat System — MVP Blueprint

Ordered plan to a working, deliverable MVP (core chat + voice)

Repository: https://github.com/HJB-2000/chatsystem
Commit audited: `e62e910e` (“connect the front and back”)
Date: 17/10/2026
Baseline: audit of 17/10/2026 (REPORT.pdf) · Auth: owned by teammate, JWT contract · Subject: ft_transcendence (modular)

Repo `chatsystem` @ `e62e910e`. Audit: `REPORT.md` (33 TODOs, IDs like P0-1). This blueprint is the ordered action list to finish the chat part of the team's ft_transcendence project. It was written against the repo and the subject (version 21.2).

Fixed decisions, not reopened: keep the two services (`chatsystem` REST + `gateway` WS), keep Kafka, auth is owned by a teammate, MVP = core chat + voice.

---

0. One-page summary

What "done" means (Definition of Done)

All of these must be true on a fresh clone, on the evaluation machine:

1. **One command**: `cp .env.example .env && docker compose up --build` brings up postgres, redis, kafka, migrations, chatsystem, gateway, the React app and the HTTPS proxy. No other command is needed (subject III.2: "run with a single command"). The `docker compose` file may be the team's; the chat services must plug into it.
2. **HTTPS only from the browser**: `https://localhost:8443` serves the SPA, `/api`, `/auth`, and `wss://…/gateway`. No `http://` or `ws://` call leaves the browser (subject III.3: "Any connection to the backend... must use HTTPS").
3. **Chrome console is clean** (no errors, no warnings) while: logging in, opening rooms, sending, editing, deleting, DMs, voice join/leave (subject III.2).
4. **No secrets in git**: `.env` untracked, `.env.example` committed (subject III.3).
5. **Validation both sides**: bad input returns 400/404/403, never 500; the React forms validate before sending (subject III.3).
6. **Identity is verified everywhere**: no request or socket frame carries a client-supplied user id. Spoof test returns 401/403.
7. **Chat features work live between two browsers**: guilds ("collectives") and rooms, invites, room messages, DMs, edit/delete, friends (request/accept/remove), presence, typing, unread on DMs, and **voice** (2–4 people hear each other).
8. **Multi-user** (subject III.2): 3 browsers open at once, no lost or duplicated message, order by `message_id`.
9. **Chat is not responsible for passwords** (auth teammate owns sign-up, hashing, login, OAuth).
10. **Privacy Policy and Terms of Service** pages exist, are not placeholders, and are linked from the chat footer. If the team has one app shell these two pages are the team's. If not, chat ships them (step 3.2).
11. **5 key tests pass** + an e2e smoke script (step 5.1, 5.2) and a demo script (step 5.4).
12. Chat README section covers what subject chapter VI requires about chat (stack, schema, features, who did what).

Subject modules this chat work supports

Names and points are quoted from the subject. A separate report covers the full matrix; this is only the pointer.

Subject module (quoted)	Points	Chat provides	Depends on auth teammate?
Web: "Major: Implement real-time features using WebSockets or similar technology."	2	gateway, live messages/typing/presence/voice signaling, reconnect	Yes, token on WS
Web: "Major: Allow users to interact with other users." (basic chat, profile, friends)	2	rooms + DMs, public profile view, friends list	Yes, needs login
Web: "Major: Use a framework for both the frontend and backend."	2	React + NestJS	No
Web: "Minor: Use an ORM for the database."	1	Prisma	No
DevOps: "Major: Backend as microservices."	2	chatsystem / gateway / auth as separate services, REST + Kafka	Needs auth as its own service
Gaming and UX: "Minor: Advanced chat features"	1	profile from chat, history persistence, typing indicators, read receipts only if you add them. Block-user is deferred. The game-dependent bullets need a game, so claim this module only if the evaluator accepts that	Yes
User Management: "Major: Standard user management and authentication."	2	not chat's, but "friends + online status" overlaps with chat. Agree who owns the friends API (this plan: chat)	Owned by auth teammate

Voice (WebRTC) does not map cleanly to any listed module. Do NOT claim "Minor: Voice/speech integration for accessibility or interaction" (AI) unless the team can justify it. Treat voice as a feature of the real-time module, or as part of a "Modules of choice" justification the team writes in the README.

Phases at a glance



Figure 1 Phase roadmap

Total about 14 working days for one developer (section 11).

Target architecture

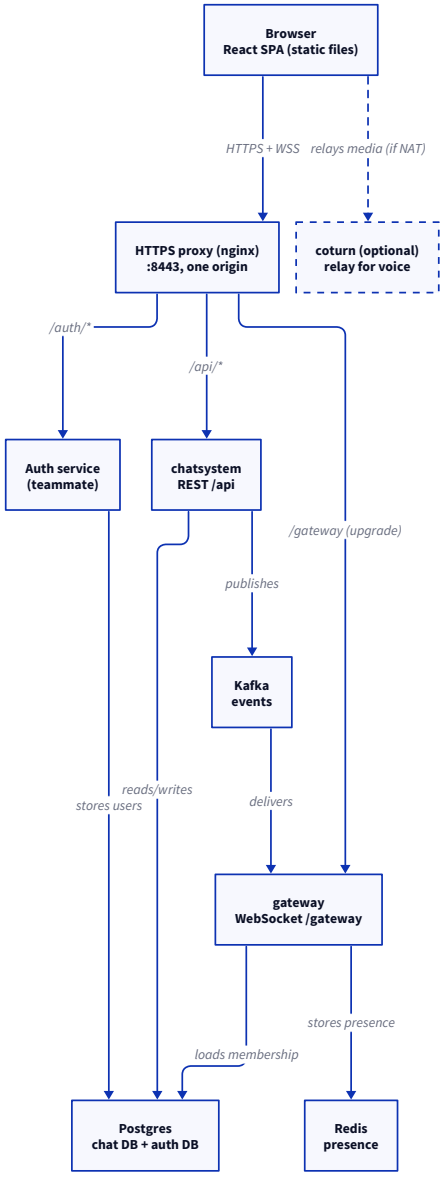


Figure 2 Target MVP architecture

One public origin (the proxy), so **no CORS** in production. Only the proxy publishes a port. Postgres, Redis and Kafka are internal to the Docker network.

---

1. Conventions and decisions used by every step

Vocabulary stays as in the repo: **collective = guild**, **room = channel**, `userId`/ids are **strings on the wire** (BigInt in DB).

Decision	Choice	Why
Token	`Authorization: Bearer <jwt>` on REST; same JWT in WS IDENTIFY `d.token`	already how IDENTIFY works; nothing new
Algorithm / secret	HS256, env `JWT_SECRET` (≥ 32 chars) shared by auth, chatsystem, gateway	simplest; no key distribution. Swap to RS256 later = change one function
`user` table owner	**Each service owns its own DB/tables.** Auth owns credentials, email, OAuth. Chat keeps a *profile mirror* table `user(user_id, username, avatar_url, …)` whose `user_id` = the token's `sub`. Chat **upserts the row from the token on first use** (REST guard and WS identify)	two Prisma schemas on one shared table = migration fights. Upsert = zero coordination at runtime. Needs: `sub` must be an integer string
Where the token lives in the browser	`localStorage['access_token']`, written by the login page (team's), read by chat	one origin, shared storage. Accepted XSS trade-off for an MVP; React escapes by default
Token expiry	access token 1 h. REST 401 → redirect to login. WS: gateway closes with **4011** at `exp`; client goes to login	no refresh flow needed for MVP; if auth ships `POST /auth/refresh`, the client uses it first
Voice	WebRTC mesh, cap 6, signaling over the existing gateway socket, **delete `apps/voice-signaling`**	one fewer service, auth already done on the gateway socket, nothing else can relay for free
Voice state	in gateway memory, never Kafka	needs latency and is ephemeral; single gateway instance (documented)
Presence	gateway + Redis only; REST stops returning it	one source of truth
DB shape	one `Channel` table (`guild_id` nullable, `type` TEXT/VOICE/DM)	removes the shared-sequence hack, the two-FK bug, and the DM race
Reconnect	client **refetches the open room's last page on every READY after a reconnect**; gateway replay buffer is a bonus, not relied on	simplest correct resync

API surface after P2 (the only routes that should remain)

All under `/api`, all need a Bearer token except where marked.

```
GET    /me                       PATCH /me
GET    /users/:id                GET /users?username=<exact>      (exact match, 1 result)
GET    /collectives              POST /collectives {name}
GET    /collectives/:id/rooms    POST /collectives/:id/rooms {name,type}
POST   /collectives/:id/invites  POST /invites/:code/join         DELETE /invites/:code
GET    /rooms/:id/members
GET    /rooms/:id/messages?limit&before      POST /rooms/:id/messages {content}
PATCH  /rooms/:id/messages/:mid {content}    DELETE /rooms/:id/messages/:mid
GET    /dms                      GET /dms/:partnerId      POST /dms/:partnerId {content}
PATCH  /dms/:partnerId/read
GET    /friends                  POST /friends {userId}   POST /friends/:id/accept   DELETE /friends/:id
GET    /voice/ice
POST   /dev/token {userId,username}      PUBLIC, only when NODE_ENV != production
```

Removed: `rooms/:id/join`, every `channels/*` alias, `GET /users`, `/users/:id/presence`, `gateway/url`, `join-voice`/`leave-voice`.

Real-time protocol after P2/P4

Client→server ops (existing): 1 heartbeat, 2 IDENTIFY, 3 presence, 8 typing, 11 ack. New: 20 `VOICE_JOIN {channel_id}`, 21 `VOICE_LEAVE`, 22 `VOICE_SIGNAL {to, kind, payload}`, 23 `VOICE_MUTE {muted}`.

Server→client events (`t`): `READY`, `MESSAGE_CREATE`, `MESSAGE_UPDATE`, `MESSAGE_DELETE`, `PRESENCE_UPDATE`, `TYPING_START`, `MEMBERSHIP_UPDATE`, `VOICE_STATE_UPDATE`, `VOICE_PEERS`, `VOICE_PEER_JOIN`, `VOICE_PEER_LEAVE`, `VOICE_SIGNAL`, `VOICE_ERROR`.

Close codes: existing 4001–4010 plus new `4011 TOKEN_EXPIRED`. The client must **not** auto-retry on 4004 or 4011.

PHASE 0 — Repo cleanup and one-command Docker stack (about 11 h)

Step 0.1 — Stop tracking junk, rotate the password

- Closes: **P0-4, P1-15** (B-23, B-37, F-13, R-05)
- Files: `.gitignore`, git index, local `.env`
- Change:
- Rotate the Postgres password first (new value only in the local, untracked `.env`).
- Replace `.gitignore` with the audit's ready-to-paste block (REPORT §7) and add `frontend/dist/`, `*.pem`.
- Run the untrack commands from REPORT §7 (`git rm -r --cached node_modules dist generated .env …`, and the react `node_modules` / `dist`), commit "chore: stop tracking deps, build output and .env".
- Optional: `git filter-repo` to shrink `.git`. Skip if the team forbids force-push on a shared repo.
- Acceptance: `git ls-files | grep -c node_modules` → `0`; `git status` clean after `npm ci && npm run build`; `git ls-files .env` empty.
- Effort: 1 h

Step 0.2 — Delete dead code and dead dependencies

- Closes: **P2-5** (B-32, B-35, B-36, R-11), part of **P1-14** (voice stub), F-12, F-13
- Files:
- delete: `apps/voice-signaling/`, `libs/` (all 7), `PyTest/`, `some_test.js`, `some_test.ts`, `tmp.md`, `backend.md`, `architecture.md`, `front-end/Api.js script.js render.js mock.js style.css index.html tailwind.config.js files.zip`, `front-end/react replacement/src/react-handbook.pdf`
- `git mv "front-end/react replacement" frontend` (a space in a path breaks Docker `COPY` lines)
- `nest-cli.json`, `tsconfig.json`, `jest.config.ts`: remove `voice-signaling` project and every `@app/*` / `libs/*` path
- `package.json`: remove `argon2 cassandra-driver socket.io @nestjs/platform-socket.io @nestjs/passport passport passport-jwt @types/passport-jwt @nestjs/config @nestjs/observe class-validator?` (keep `class-validator` and `class-transformer`, step 2.2 uses them; keep `@nestjs/jwt`). Remove script `start:dev:voice`.
- keep the architecture images if the team wants them, but move to `docs/`
- Acceptance: `npm ci && npx nest build chatsystem && npx nest build gateway` both succeed; `grep -r "voice-signaling\|@app/" apps nest-cli.json tsconfig*.json` returns nothing.
- Effort: 1.5 h

Step 0.3 — One `.env.example`, consistent ports

- Closes: **P2-4** (B-15, R-06), part of **P0-4**
- Files: `.env.example` (new, root), delete `infra/postgres/env.example` and `infra/postgres/README.md` after merging content, `apps/chatsystem/src/main.ts:8` (remove the `client` static dir)
- Change: one file, no duplicated keys: `` `POSTGRES_USER=chat POSTGRES_PASSWORD=change-me POSTGRES_DB=chat DATABASE_URL=postgresql://chat:change-me@postgres:5432/chat REDIS_URL=redis://redis:6379 KAFKA_BROKERS=kafka:9092 CHAT_PORT=3000 GATEWAY_PORT=3001 JWT_SECRET=change-me-to-32+random-chars NODE_ENV=development HTTPS_PORT=8443 STUN_URLS=stun:stun.l.google.com:19302 TURN_URL= TURN_USER= TURN_PASS= VOICE_MAX_PEERS=6 ` `` main.ts: `delete` useStaticAssets; `listen on` process.env.CHAT_PORT ?? 3000 `only. Gateway listens on` GATEWAY_PORT ?? 3001 `only (drop the` PORT`` fallbacks).
- Acceptance: `grep -c "^REDIS_URL" .env.example` → `1`; `CHAT_PORT=3000 GATEWAY_PORT=3001 node dist/apps/chatsystem/main` and the gateway start side by side without a port clash.
- Effort: 0.5 h

Step 0.4 — Dockerfiles (backend ×2 apps, frontend)

- Closes: part of **P2-11** (B-40), support for **P0-4**
- Files: `Dockerfile` (new, root, one file for both Nest apps), `frontend/Dockerfile` (new), `.dockerignore` (new), `package.json` (`postinstall: prisma generate`, scripts `start:chat`, `start:gateway`)
- Change:
- Root `Dockerfile` with `ARG APP` (`chatsystem` or `gateway`): `` `FROM node:24-slim AS build WORKDIR /app COPY package*.json prisma.config.ts ./ COPY prisma ./prisma RUN npm ci COPY . . ARG APP RUN npx prisma generate && npx nest build $APP FROM node:24-slim WORKDIR /app ENV NODE_ENV=production COPY --from=build /app /app ARG APP ENV APP=$APP CMD ["sh","-c","node dist/apps/$APP/main.js"] ` `` Keeping the full node_modules `in the final image is a deliberate simplification (also needed for` prisma migrate deploy`). Debian-slim, not alpine, to avoid musl issues with native packages.
- `frontend/Dockerfile`: `node:24-slim` build stage (`npm ci && npm run build`) → `nginx:1.27-alpine` serving `dist/` with `try_files $uri /index.html`.

- `.dockerignore`: `node_modules dist generated .git .env frontend/node_modules`.
- Make sure the `prisma.config.ts` `datasource.url` can be read at `migrate deploy` time (`DATABASE_URL` is in the container env).
- Acceptance: `docker build --build-arg APP=chatsystem -t chat-api .` and the same for `gateway` and `docker build -t chat-web frontend` all succeed.
- Effort: 2 h

Step 0.5 — `docker-compose.yml` with healthchecks, migrations, Kafka listeners

- Closes: **P0-4** (ports), **P2-3** infra part, supports **P1-6**
- Files: `docker-compose.yml` (new, root; the old `infra/postgres/docker-compose.yml` is deleted), `infra/postgres/init/01-init.sql` (keep, mounted)
- Change:
- Services: `postgres` (16-alpine, healthcheck `pg_isready`), `redis` (7-alpine, healthcheck), `kafka` (apache/kafka 3.8.0, healthcheck as today), `migrate`, `chatsystem`, `gateway`, `web`, `proxy` (step 0.6).
- **No `ports:` for postgres/redis/kafka** by default (they are internal). For local debugging add a `docker-compose.override.yml` (git-ignored) that publishes them on `127.0.0.1:` only.
- **Kafka needs two listeners**, or containers cannot reach it (the current `advertised.listeners=localhost:9092` only works from the host): `` `KAFKA_LISTENERS: INTERNAL://:9092,EXTERNAL://:29092,CONTROLLER://:9093 KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:9092,EXTERNAL://localhost:29092 KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL` Keep the other `KAFKA_*` lines from the current file.
- `migrate`: same image as chatsystem, `command: npx prisma migrate deploy`, `depends_on: postgres: service_healthy`, `restart: "no"`.
- `chatsystem` and `gateway`: `build: {context: ., args: {APP: …}}`, `env_file: .env`, `restart: unless-stopped`, `depends_on: migrate: {condition: service_completed_successfully}, kafka: {condition: service_healthy}, redis: …`. Healthcheck: `node -e "fetch('http://localhost:3000/api/health').then(r=>process.exit(r.ok?0:1))"` (add `GET /api/health` public, returns `{ok:true}`; the gateway uses `/gateway/health`).
- `restart: unless-stopped` also covers the case where Kafka is slow and the gateway crashes in `onModuleInit` (`apps/gateway/src/app/kafka.service.ts:198-213`).
- Acceptance: `docker compose up -d --build && docker compose ps` shows every service `healthy` (or `exited 0` for `migrate`) within ~90 s; `docker compose ps --format '{{.Ports}}' | grep -E '5432|6379|9092'` prints nothing.
- Effort: 3 h

Step 0.6 — HTTPS reverse proxy (one origin)

- Closes: subject III.3 (HTTPS); prerequisite for voice (`getUserMedia`)
- **Use the team's existing proxy if there is one** (ask the devops teammate on day 1). Chat then only needs three `location` blocks, shown below. If there is no proxy yet, ship this one under compose profile `standalone`, and the team can delete it later.
- Files: `infra/proxy/nginx.conf` (new), `infra/proxy/entrypoint.sh` (new), `docker-compose.yml` (service `proxy`)
- Change:
- `proxy`: `nginx:1.27-alpine`, `ports: ["${HTTPS_PORT:-8443}:8443"]`, entrypoint generates a self-signed cert if missing: `openssl req -x509 -nodes -newkey rsa:2048 -days 365 -keyout /certs/key.pem -out /certs/cert.pem -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"` (install `openssl` with `apk add` in a small Dockerfile).
- The three blocks the team's proxy needs: `` `location /api/ { proxy_pass http://chatsystem:3000; } location /gateway { proxy_pass http://gateway:3001; proxy_http_version 1.1; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"; proxy_read_timeout 3600s; } location / { proxy_pass http://web:80; }` The auth teammate adds location /auth/ { proxy_pass http://auth:PORT; }`.
- Optional: `limit_req_zone` on `/api/` (rate limit belongs in the proxy).
- Acceptance: `curl -sk https://localhost:8443/api/health` → `{"ok":true}`; `curl -s http://localhost:8443` fails (no plain HTTP); `wscat -n -c wss://localhost:8443/gateway` prints a `HELLO` frame (`op:10`).
- Effort: 2 h

Step 0.7 — Lint, start scripts, smoke of the whole stack

- Closes: **P2-1** (R-04, B-39), **P2-11** (B-40), **P2-12** (timeboxed)
- Files: `package.json`
- Change: `"lint": "oxlint apps"`; `"start:prod": "node dist/apps/chatsystem/main"` and `"start:gateway": "node dist/apps/gateway/main"`; pin `engines.node` to `>=24`. Timebox 30 min to find the `TimeoutNegativeWarning` source (grep for `setTimeout` / `setInterval` computed from `Date.now()`); if not found, leave it and note it in the README known issues.
- Acceptance: `npm run lint` exit 0 and lists files; `docker compose down -v && docker compose up --build -d` from scratch reaches all healthy; `curl -sk https://localhost:8443/` returns the React `index.html`.
- Effort: 1 h

PHASE 1 — Auth integration via the contract (about 8 h, plus the teammate's work in parallel)

Hand the **"Contract for the auth teammate" (section 9)** to the teammate on day 1. The dev stub (step 1.1) means chat work is never blocked.

Step 1.1 — JWT verify, global guard, `@CurrentUser()`, dev token

- Closes: **P0-1** (a, b, f server half), B-04
- Files (all new unless noted): `apps/chatsystem/src/auth/jwt.ts`, `auth.guard.ts`, `current-user.decorator.ts`, `public.decorator.ts`, `dev-token.controller.ts`; `apps/chatsystem/src/app.module.ts` (register `APP_GUARD`, `JwtModule.register({})`)
- Change:
- `jwt.ts`: `verifyAccessToken(token)` using `JwtService.verify(token, { secret: process.env.JWT_SECRET, algorithms: ['HS256'] })`, then checks `/^\d{1,18}$/.test(sub)` and a non-empty `username`. Returns `{ userId: bigint, username, avatar?, exp }`. **Always pass `algorithms`** (blocks `alg:none` and key-confusion).
- Guard: read `Authorization: Bearer …` → verify → `prisma.user.upsert({ where:{userId}, update:{username}, create:{userId, username} })` → `req.user = {id: string, username}`. No/invalid token → `UnauthorizedException`. Skip routes marked `@Public()`.
- On boot: throw if `JWT_SECRET` is missing or shorter than 32 chars.
- `DevTokenController` (`POST /api/dev/token {userId, username}`) signs a 1 h HS256 token. The module registers it only if `process.env.NODE_ENV !== 'production'`; it logs a loud WARN at boot.
- Schema note: for the upsert to work, `User.userId` must not be `autoincrement()`. That is changed in step 2.1; until then the dev token only accepts ids that exist in the seed (1–3).
- Acceptance: `` `curl -sk -o /dev/null -w '%{http_code}\n' https://localhost:8443/api/me # 401 TOKEN=$(curl -sk -X POST https://localhost:8443/api/dev/token \ -H 'content-type: application/json' -d '{"userId":"1","username":"alice"}' | jq -r .token) curl -sk https://localhost:8443/api/me -H "Authorization: Bearer $TOKEN" # 200, id "1"` ``
- Effort: 2.5 h

Step 1.2 — Remove every client-supplied identity

- Closes: **P0-1** (c, d), **P0-2** (caller part), B-01, B-21, B-22
- Files: `apps/chatsystem/src/app.controller.ts` (all routes), `apps/chatsystem/src/chat.service.ts` (signatures), `frontend/src/api/chatApi.ts` later in 3.2
- Change: every handler takes `@CurrentUser() user` and passes `user.id`. Delete from DTOs/queries: `userId`, `senderId`, `:userId` in DM routes. `PATCH users/:id` → `PATCH /api/me` (own row only). `getCollectives`, `getRoomMembers`, `getRoomMessages` etc. use `user.id`. Do the full route rename here (table in section 1) except the new friends/DM behavior, which comes in P2.
- Acceptance (the spoof test from the audit now fails): `` `curl -sk -X POST https://localhost:8443/api/rooms/1/messages -H "Authorization: Bearer $TOKEN_ALICE" \ -H 'content-type: application/json' -d '{"content":"hi","senderId":"3"}' # 201, and GET shows senderId == "1" (alice), not "3". Same call without token: 401. curl -sk -X PATCH https://localhost:8443/api/users/2 … # 404 (route gone)` ``
- Effort: 3 h

Step 1.3 — Gateway verifies the token, enforces expiry and frame size

- Closes: **P0-1** (e), **P0-5** (B-26), B-04
- Files: `apps/gateway/src/app/session-loader.service.ts:60-103`, `apps/gateway/src/app/protocol.ts:12-20`, `apps/gateway/src/app/gateway.ts:139-198`, `apps/gateway/src/main.ts:8`, `apps/gateway/src/app/info.controller.ts`, `apps/gateway/src/app/jwt.ts` (new, same 15 lines as chat; duplicating is simpler than a shared lib), `apps/gateway/src/gateway.module.ts`
- Change:
- `SessionLoader.load(token)`: `verifyAccessToken` (delete `ID_RE.test(token)`), then `user.upsert` (same as the guard) so a user who never called REST still has a row. Return `exp`.
- `Close.TOKEN_EXPIRED = 4011`. On identify, `setTimeout(() => ws.close(4011,'token expired'), Math.min(exp*1000 - Date.now(), 2**31-1))`; clear it in `handleDisconnect`.
- `new WsAdapter(app, { maxPayload: 16 * 1024 })` (16 KiB, not 4 KiB: SDP offers in step P4 are 2–4 KB).
- Cap sessions per user at 5 (close the oldest with 4008 or refuse).
- `GET /gateway/stats`: restrict to `NODE_ENV !== 'production'` or require a header `x-admin-key` from env. Delete `POST /gateway/url` (React uses same-origin now).
- Remove `app.enableCors()` in `gateway/main.ts` (same origin through the proxy).

- Acceptance: `` `wscat -n -c wss://localhost:8443/gateway > {"op":2,"d":{"token":"2"}} # closed 4004 (the old "token == id" no longer works) > {"op":2,"d":{"token":"<valid>"}} # READY with user.id == token's sub` Send a 20 KB frame → connection closes (1009). A token with exp 10 s ahead (/dev/token accepts optional ttl`) gets closed with 4011 after 10 s.
- Effort: 2.5 h

Step 1.4 — Seed through the same path; REST rate limit

- Closes: **P0-5** (REST half)
- Files: `prisma/seed.ts`, `apps/chatsystem/src/app.module.ts`, `package.json` (`@nestjs/throttler`)
- Change: seed creates users with explicit ids 1–3 (matching the dev token). Add `ThrottlerModule.forRoot([{ ttl: 60_000, limit: 120 }])` and `ThrottlerGuard` as a second `APP_GUARD` (after the auth guard). Rate limiting at the proxy is the recommended place too; both are cheap.
- Acceptance: 130 rapid `GET /api/me` calls → the last ones return 429; `npm run db:seed` re-runs without error.
- Effort: 1 h

PHASE 2 — Backend correctness (about 24 h)

Step 2.1 — Final schema, squashed migration

- Closes: **P2-3** (B-10, B-11, R-08), **P2-13** (B-16, B-35), **P2-7** (race half), prepares P1-11, P1-12, P1-14
- Files: `prisma/schema.prisma`, `prisma/migrations/*` (delete all 3, generate one `init`), `prisma/seed.ts`
- Change (no real data exists, so squash is safe):
- `User`: `userId BigInt @id` (**no autoincrement**, it equals the token `sub`), keep `username @unique`, `avatarUrl?`, `roleTitle?`, `currentActivity?`. **Remove** `lastPresence`.
- `Channel`: `guildId BigInt?`, `type ChannelType` = `TEXT | VOICE | DM` (drop `CATEGORY`), add `dmKey String? @unique` (`"<lowId>:<highId>"`), drop `capacity`, `categoryId`, `icon` if unused. Drop `DirectChannel` model. Keep `DirectChannelMember(channelId,userId)`.
- `Message`: single relation `channel Channel @relation(…, onDelete: Cascade)`; drop `embeds`, `attachments`; `@@index([channelId, messageId])` stays.
- `Member`: drop `roleIds`, `voiceChannelId`; drop the `Role` model. Drop `Guild.memberCount` and compute with `_count` (it was never decremented).
- `Friendship`: add `status FriendStatus @default(PENDING)` (`PENDING | ACCEPTED`).
- New `ChannelRead(channelId, userId, lastReadMessageId, @@id([channelId,userId]))`.
- Index `direct_channel_member(channel_id)` is already covered by the composite PK `(channel_id, user_id)`; drop the redundant `@@index([channelId])`.
- Delete the migrations folder, run `npx prisma migrate dev --name init`, commit the single migration.
- Acceptance: `docker compose down -v && docker compose up -d` applies one migration; `npx prisma migrate diff --from-config-datasource --to-schema prisma/schema.prisma --exit-code` → exit 0 (no drift); `npm run db:seed` works.
- Effort: 3 h

Step 2.2 — Validation, BigInt-safe responses, error mapping, CORS env

- Closes: **P1-7** (B-19, B-20, R-03), part of **P1-8** (B-27, B-28 origin), B-36 (duplicate routes)
- Files: `apps/chatsystem/src/main.ts`, `apps/chatsystem/src/dto/*.ts` (new), `apps/chatsystem/src/common/parse-bigint.pipe.ts` (new), `apps/chatsystem/src/common/prisma-exception.filter.ts` (new), `apps/chatsystem/src/app.controller.ts`, `apps/chatsystem/src/chat.service.ts:333-343`
- Change:
- `app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }))`.
- DTOs with `class-validator`: `content` string 1–4000 trimmed; `name` 1–50; `username` 3–32; `type` in `TEXT|VOICE`.
- `ParseBigIntPipe`: accepts `/^\d{1,18}$/`, else `BadRequestException('invalid id')`. Use on every `:id` param.
- Exception filter: Prisma `P2002` → 409, `P2003` → 400, `P2025` → 404.
- `BigInt.prototype.toJSON = function () { return this.toString(); }` at the top of `main.ts` as a safety net; keep shaping IDs to strings explicitly in `shapeMessage` and friends.
- `editMessage`: reuse the same 1–4000 check as create.
- CORS: `enableCors({ origin: process.env.CORS_ORIGIN?.split(','), credentials: false })`. In production there is one origin so you may disable CORS entirely; keep it only for the Vite dev server.
- Delete the `channels/*` duplicate routes and the `senderId ?? userId`, `message ?? content` aliases (one field: `content`).
- Add the throttler's `@SkipThrottle()` on `/api/health`.
- Acceptance: `` `curl -sk -o /dev/null -w '%{http_code}\n' https://localhost:8443/api/users/abc -H "Authorization: Bearer $T" # 400 curl -sk … -X POST …/rooms/1/messages -d '{}' # 400 (was 500) curl -sk … -X POST …/rooms/1/messages -d '{"content":"'$(head -c 5000 /dev/zero | tr '\0' a)'"}' # 400 curl -sk … -X DELETE …/rooms/1/messages/99999999999999 # 404` ``
- Effort: 3.5 h

Step 2.3 — Kafka producer: lazy connect + retry

- Closes: **P1-6** (B-07, B-17, B-18)
- Files: `apps/chatsystem/src/kafka.service.ts:22-48`
- Change: replace the connect-once-at-boot logic: `` `ts private connecting?: Promise<void>; private async ensure() { if (this.connected) return; this.connecting ??= this.producer.connect() .then(() => { this.connected = true; }) .finally(() => { this.connecting = undefined; }); await this.connecting; } async publish(topic: string, key: string, value: unknown) { for (let i = 0; i < 4; i++) { try { await this.ensure(); await this.producer.send({ topic, messages: [{ key, value: JSON.stringify(value) }] }); return true; } catch (e) { this.connected = false; await this.producer.disconnect().catch(() => undefined); await new Promise(r => setTimeout(r, 200 * 2 ** i)); // 200,400,800,1600 ms } } this.log.error(publish ${topic} gave up); return false; // never throws } ` `` Remove the onModuleInit `connect (or keep it as best-effort, ignoring failure). Call sites keep` void this.kafka.publish(...): `the`

`message is already stored, the push must not block the HTTP response. Keep the producer's` retry` default. The client already refetches on reconnect (step 3.3), so a dropped push is repaired by the next resync.

- Acceptance: `docker compose stop kafka`, post a message (201 returns), `docker compose start kafka`, wait ~30 s, post a second message → a WS client receives the second one live. Also: `docker compose up` order does not matter (start chatsystem before kafka is healthy via `docker compose up -d --no-deps chatsystem kafka`, then post).
- Effort: 1.5 h

Step 2.4 — Edit/delete events end to end (gateway side)

- Closes: **P1-3** (B-05), server half
- Files: `apps/gateway/src/app/protocol.ts:22-28` (add `MESSAGE_UPDATE`, `MESSAGE_DELETE`, `MEMBERSHIP_UPDATE` to `Evt`), `apps/gateway/src/app/events.consumer.ts:19-31`, `apps/chatsystem/src/chat.service.ts` (payloads)
- Change: in `onMessage` switch on `v.type`: `undefined | 'MESSAGE_CREATE'` → `Evt.MESSAGE_CREATE` + `registry.remember(...)`; `'MESSAGE_EDIT'` → `Evt.MESSAGE_UPDATE` with `{channel_id, message_id, content, edited_at}`; `'MESSAGE_DELETE'` → `Evt.MESSAGE_DELETE` with `{channel_id, message_id}`. Do **not** put update/delete into the replay buffer. Set `type: 'MESSAGE_CREATE'` explicitly in the create payload (`chat.service.ts:~315`). Also verify `editMessage` / `deleteMessage` check `message.channelId === roomId` and the caller can still read the channel.
- Acceptance: two `wscat` sessions in room 1. `PATCH …/messages/<id> {"content":"edited"}` → both get `MESSAGE_UPDATE` (not CREATE). `DELETE` → both get `MESSAGE_DELETE`. Another user's PATCH → 403.
- Effort: 1.5 h

Step 2.5 — DMs: live delivery, one channel per pair, friendship required

- Closes: **P1-1** (B-02), **P0-2** (B-22), **P2-7** (DM race)
- Files: `apps/chatsystem/src/chat.service.ts:411-478`, `apps/chatsystem/src/app.controller.ts` (DM routes), `apps/gateway/src/app/session-loader.service.ts:70-102`
- Change:
- `getOrCreateDm(me, partner)`: require an `ACCEPTED` friendship (else 403). Compute `dmKey = min:max`. `prisma.channel.upsert({ where:{dmKey}, create:{type:'DM', name:'dm', dmKey, …, members: createMany}, update:{} })` inside a try/catch that retries once on `P2002`. No more check-then-create.
- Routes: `GET /dms` (list of accepted friends with last message and unread; **do not create DMs** while listing, `GET dms/:userId` used to create one per friend), `GET /dms/:partnerId` (history, page by `before`), `POST /dms/:partnerId {content}` (creates the channel lazily, calls the same create path as rooms).
- `assertCanReadChannel`: for a `DM` channel require a `DirectChannelMember` row for the caller.
- Gateway loader: also read `directChannelMember` rows for the user and add each `channelId` to `channels` (guild id value `''`).
- After creating a DM channel, publish a membership event for both users (step 2.7) so an already-open socket learns it.
- Acceptance: alice and bob are friends (seed or step 2.6). Bob's `wscat` is open. `POST /api/dms/2` as alice → bob receives `MESSAGE_CREATE` with a DM `channel_id`. Carol (not a party) `GET /api/rooms/<dmChannelId>/messages` → 403. Alice `POST /api/dms/3` without friendship → 403.
- Effort: 3 h

Step 2.6 — Friends: request, accept, remove, list

- Closes: **P1-13** (friends part), **P0-2** (forced friendship)
- Files: `apps/chatsystem/src/chat.service.ts:481-499`, `apps/chatsystem/src/app.controller.ts`, `apps/chatsystem/src/dto/friend.dto.ts` (new)
- Change: `GET /users?username=` (exact match, returns `[{id, username, avatarUrl}]`, max 1) replaces the old `GET /users` list (P1-8). `POST /friends {userId}`: creates `(me → them, PENDING)`; refuses self and duplicates; if `them → me` is already PENDING, accept both. `POST /friends/:id/accept`: only the addressee; sets `ACCEPTED` and upserts the reverse row `ACCEPTED` in one `$transaction`. `DELETE /friends/:id`: deletes both rows. `GET /friends` returns `{friends:[…], incoming:[…], outgoing:[…]}`. Publish a membership event to both users on accept.
- Acceptance: alice `POST /friends {"userId":"2"}` → 201; bob `GET /friends` shows alice in `incoming`; bob `POST /friends/1/accept` → both lists show each other under `friends`; alice `DELETE /friends/2` → gone for both; bob `POST /friends/1/accept` as a third user → 403/404.
- Effort: 2.5 h

Step 2.7 — Membership refresh on a live socket

- Closes: **P1-2** (B-03), supports P1-13 and P1-1
- Files: `apps/chatsystem/src/chat.service.ts` (publish points), `apps/gateway/src/app/config.ts:12` (`TOPICS.membership`), `apps/gateway/src/app/events.consumer.ts`, `apps/gateway/src/app/registry.service.ts:45-76`, `apps/gateway/src/app/gateway.ts`
- Change:

- Chat publishes `{ user_ids: ["1","2"] }` on topic `membership` after: create guild (owner), create room (all guild members), join by invite (the joiner), friend accept (both), DM channel created (both). Never block the request (`void`).
- Gateway consumer: for each `user_id` that has a local session, call `loader.load`-equivalent (a `loadMemberships(userId)` split from `load(token)`), then `registry.refreshUser(userId, state)`.
- `registry.refreshUser`: **drop the user from all four indexes, then re-add from the new state, and update every session of that user** (`s.channels`, `s.guildIds`, `s.friendIds`). Fix the early return at `registry.service.ts:50` that makes a second socket of the same user skip indexing. Then send `MEMBERSHIP_UPDATE {guild_ids, channel_ids}` to that user's sessions.
- Register `TOPICS.membership` in the gateway topic list (the gateway creates topics at boot, `kafka.service.ts:199-209`).
- Acceptance: bob is connected via `wscat`. Alice creates a room in a guild bob is in → bob receives `MEMBERSHIP_UPDATE`, then a message posted in the new room reaches bob **without reconnecting**. Same for: bob joins a new guild by invite, then receives its messages.
- Effort: 3 h

Step 2.8 — Presence: one source (gateway + Redis)

- Closes: **P1-12** (B-12, B-13)
- Files: `apps/chatsystem/src/chat.service.ts:47-68, 388-409` (remove `setPresence` and every `lastPresence` read), `apps/gateway/src/app/presence.service.ts`, `apps/gateway/src/app/events.consumer.ts:136-147`, `apps/gateway/src/app/session-loader.service.ts`, `apps/gateway/src/app/gateway.ts:176-191, 84-85`
- Change:
- Load `peerIds` in the loader: distinct `userId` from `member` where `guildId in user's guilds`, plus accepted friends, capped at 2000.
- READY carries `online: [{user_id,status}]` for all `peerIds` (rename from `friends_online`).
- `presence.set(userId, status, guildIds)` publishes `{user_id, status, guild_ids}`. Consumer delivers `PRESENCE_UPDATE` with `registry.toGuild` for each guild id plus `toWatchers`. Including `guild_ids` in the payload is what makes **offline** reach the guild (the session is already removed when offline is published).
- On the last local session closing → set `offline` (existing). On heartbeat → `touch`. Document: **one gateway instance**; a second instance would publish a false `offline`.
- REST: `GET /rooms/:id/members` stops returning presence; delete `PATCH /users/:id/presence` (clients send op 3 over WS).
- Acceptance: alice and bob in the same guild. Bob's socket opens → alice receives `PRESENCE_UPDATE online` for bob. Bob's socket closes → alice receives `offline`. `GET /api/rooms/1/members` has no `presence` field.
- Effort: 3 h

Step 2.9 — Remove open join, harden invites, scope reads

- Closes: **P0-3** (B-25), **P2-8** (B-29), **P1-8** (B-27, B-28)
- Files: `apps/chatsystem/src/chat.service.ts:160-168, 198-250`, `apps/chatsystem/src/app.controller.ts`
- Change: delete `POST rooms/:id/join` and `joinRoom`. Invite creation sets `expiresAt = now + 7 d` (env `INVITE_TTL_HOURS`); `joinByInvite` checks it (410 Gone if expired); add `DELETE /invites/:code` for the creator. `getCollectiveRooms` throws 403 for non-members (the old code returned room previews with `lastMessage` of private guilds). `GET /users/:id` returns only `{id, username, avatarUrl, roleTitle, currentActivity}`.
- Acceptance: `curl -sk -X POST …/api/rooms/1/join` → 404. A user who is not in guild 1: `GET /api/collectives/1/rooms` → 403. An invite with `expiresAt` in the past → 410; a valid invite makes the user a member and the member appears in `GET /rooms/:id/members`.
- Effort: 2 h

Step 2.10 — Room previews and DM unread

- Closes: **P1-10** (B-08), **P1-11** (B-09)
- Files: `apps/chatsystem/src/chat.service.ts:84-93, 109, 425-427`, `apps/chatsystem/src/app.controller.ts` (`PATCH dms/:partnerId/read`), `apps/gateway/src/app/gateway.ts:223-227`
- Change: the `messages` relation in the room query gets `include: { author: { select: { username: true } } }`. `PATCH /dms/:partnerId/read` upserts `ChannelRead.lastReadMessageId` = latest message id. DM unread = `count(message where channelId = dm and messageId > lastRead and authorId != me)`, computed with one `groupBy` over the user's DM channels (not a loop). Delete the gateway `ACK` state (`acks`, `lastAckAt`, `onAck`) or leave op 11 as a no-op; room unread is a **client-side** counter (step 3.4).
- Acceptance: alice sends 3 DMs to bob → `GET /api/dms` as bob shows `unread: 3`; bob `PATCH /api/dms/1/read` → `unread: 0`; a room preview shows `lastMessage.author` = real username, not "Unknown".
- Effort: 2.5 h

PHASE 3 — React completion (about 32 h)

All paths are under `frontend/` (renamed in step 0.2).

Step 3.1 — Project hygiene, env config, split the monolith, remove fake UI

- Closes: **P1-5** (F-01, F-09, R-09), **P2-6** (F-14), **P2-9** (F-15, F-16), UI half of **P1-14**
- Files: `frontend/package.json`, `frontend/vite.config.ts` (new), `frontend/tsconfig.json` (new), `frontend/src/api/chatApi.ts:3-5`, `frontend/src/realtime/gatewayClient.ts:41-43`, `frontend/src/ui.jsx` (split into `frontend/src/components/*.tsx`), `frontend/index.html`
- Change:
- Pin deps with `^`, move `vite` and `@vitejs/plugin-react` to `devDependencies`, remove `mdb-react-ui-kit`, add `typescript`. Rename files to `.tsx` over time, enable `tsc --noEmit` in the `build` script.
- `vite.config.ts`: `react()`, `base: process.env.VITE_BASE ?? '/'`, dev proxy `/api` → `https://localhost:8443` (`secure:false`) and `/gateway` with `ws: true`.
- Config: `const API = import.meta.env.VITE_API_URL ?? ''` (empty = same origin) and `const WS = (location.protocol === 'https:' ? 'wss://' : 'ws://') + location.host + '/gateway'`. Delete `__NEXUS_CONFIG__`.
- Split `ui.jsx` (539 lines) into `TopBar`, `ServerRail`, `ChannelPanel`, `MessageArea`, `Composer`, `Roster`, `Dialogs/*`, `App`.
- Remove fake UI: Copilot, notifications bell, top search, "Live Study Rooms", file attachments (`ui.jsx:254`), the "Message sent" toast on every send (`ui.jsx:355`), the Unsplash images, the fake huddle (`ui.jsx:214-216, 488-508`; real voice UI comes in P4), and rename "NexusCampus" to the team's product name. Replace Material Symbols/Google Fonts links by locally installed `material-symbols` / `@fontsource` packages (avoids third-party requests and console noise).
- Acceptance: `cd frontend && npm ci && npm run build` succeeds including `tsc`; `grep -rn "__NEXUS_CONFIG__\|Copilot\|Unsplash" frontend/src` → nothing; `npm run dev` proxies `/api` to the stack.
- Effort: 4 h

Step 3.2 — Auth bootstrap, API client, footer pages

- Closes: **P0-1** (f), part of **P1-9** (auth failure handling), subject III.2 (Privacy/ToS)
- Files: `frontend/src/auth/token.ts` (new), `frontend/src/api/chatApi.ts`, `frontend/src/components/DevLogin.tsx` (new), `frontend/src/pages/Privacy.tsx`, `Terms.tsx` (new, only if the team has no shell), `frontend/src/components/Footer.tsx`
- Change:
- `token.ts`: `getToken()` reads `localStorage['access_token']` (try/catch), `isExpired()` decodes the payload with `atob` and compares `exp`, `clear()`, and a `storage` event listener so logging out in another tab logs out here.
- `chatApi.ts`: add `Authorization: Bearer ${getToken()}` to every request. On 401: `clear()` then `location.assign('/login?next=' + encodeURIComponent(location.pathname))`. Delete all `userId` / `senderId` parameters from calls. Add the new calls (create collective/room/invite, join invite, DMs, friends, edit/delete, `me`, `voice/ice`).
- No token at startup: in production redirect to `/login`; if `import.meta.env.VITE_DEV_LOGIN === 'true'`, render `DevLogin` (username+id form → `POST /api/dev/token` → store) so you never depend on the teammate.
- Remove `?userId=` handling (`ui.jsx:6-7`).
- Footer with links to `/privacy` and `/terms`. Content must be real: what data chat stores (username, messages, friend list, presence), retention, who can see what, deletion contact, acceptable use. Mirror whatever the team writes; do not diverge.
- Acceptance: with no token, the page lands on the login (or DevLogin); with a dev token, `GET /api/me` runs with the header (check Network tab); deleting the key in another tab logs this tab out; `/privacy` and `/terms` render real text.
- Effort: 3 h

Step 3.3 — Gateway client rewrite (StrictMode-safe, reconnect, resync)

- Closes: **P1-9** (F-05, F-06, F-07), supports P1-3, P1-4
- Files: `frontend/src/realtime/gatewayClient.ts`, `frontend/src/hooks/useGateway.ts` (new)
- Change:
- One `GatewayClient` instance, created in a `useEffect` with cleanup: ``ts useEffect(() => { gw.connect(getToken()!); return () => gw.disconnect(); }, []); `` In open() capture const socket = new WebSocket(url) and in every handler start with if (socket !== this.socket) return;` so the socket React StrictMode creates and immediately discards cannot call back into the live client.
- `reconnectAttempt = 0` on `READY`. Backoff `min(1000*2**n, 15000)` plus random jitter.
- `socket.onclose = (e) => { if (e.code === 4004 || e.code === 4011) { clear(); goLogin(); return; } if (!this.manuallyClosed) scheduleReconnect(); }`.
- Wrap `JSON.parse` in try/catch. Remove `lastMessageId` from `localStorage` (global across users was the bug, F-06); the cursor is no longer needed because the client refetches on reconnect. Keep sending `last_message_id` only from memory if you want the gateway's replay as a bonus.

- Emit a `reconnected` event after the second and later `READY`; `App` refetches rooms and the open room's last page then. Handle `MEMBERSHIP_UPDATE` by refetching collectives, rooms and DMs.
- Hook returns `status` (`connecting|ready|reconnecting`) so the UI shows a small banner instead of failing silently.
- Acceptance: in Chrome with the dev server (StrictMode on) the Network tab shows **one** live WS after load. Kill the gateway container for 20 s and restart it → banner "reconnecting", then back to ready, and the open room shows messages posted during the gap. Expired token → lands on login, not a reconnect loop.
- Effort: 4 h

Step 3.4 — Message list: per-room filter, ordering, history, edit/delete, unread

- Closes: **P1-4** (F-02), **P1-3** (UI half), **P2-7** (client sort), part of **P1-13**
- Files: `frontend/src/components/MessageArea.tsx`, `Composer.tsx`, `frontend/src/state/messages.ts` (new reducer)
- Change:
- Keep `roomRef` in sync with the open room; handle `MESSAGE_CREATE` as: if `channel_id !== roomRef.current.id` → increment `unread[channel_id]` (client counter, shown as a badge) and return; else insert.
- Insert is idempotent: dedupe by `id`, then **sort by BigInt**: `(a,b) => (BigInt(a.id) < BigInt(b.id) ? -1 : 1)`. Never `Number()` or string-compare ids.
- `MESSAGE_UPDATE` replaces content and sets `editedAt`; `MESSAGE_DELETE` removes by id.
- "Load older" button using `?before=<oldest id>&limit=50`.
- Own messages show Edit (inline textarea, Enter saves, Esc cancels) and Delete (confirm). Show "(edited)".
- Composer: trims, max 4000 chars with a counter, disabled while sending, shows server error text inline. Send typing op 8 at most once per 3 s while typing.
- Acceptance: browser A and B in room 1. A sends → B sees it once, in order. A edits → B sees "(edited)" without a new bubble. A deletes → B's bubble disappears. B is in room 2 while A posts in room 1 → B's room-2 view is untouched and room 1 shows an unread badge. Console has no errors.
- Effort: 5 h

Step 3.5 — Create collective and room, invites

- Closes: **P1-13** (create guild/room, invites)
- Files: `frontend/src/components/dialogs/CreateCollective.tsx`, `CreateRoom.tsx`, `InviteDialog.tsx`, `JoinInvite.tsx` (new), `ServerRail.tsx`, `ChannelPanel.tsx`
- Change: "+" in the server rail → create collective (name 1–50, client validation matches the DTO). "+" in the channel panel → create room (name, type text/voice). "Invite" button → `POST /collectives/:id/invites`, shows the code and a copy button; "Join" dialog takes a code → `POST /invites/:code/join`, then refetch. After each action rely on `MEMBERSHIP_UPDATE`, but also refetch locally so the actor sees the result immediately. Map server 400/403/404/410 to readable messages.
- Acceptance: user A creates a collective and a text room, creates an invite. User B pastes the code → the collective appears in B's rail without reload, B receives room messages live (step 2.7). An expired or bogus code shows an inline error.
- Effort: 4 h

Step 3.6 — DMs and friends

- Closes: **P1-1** (UI half, F-08), **P1-13** (friends, DM screen)
- Files: `frontend/src/components/FriendsPanel.tsx` (new), `ChannelPanel.tsx:54-170`, `App` (`onSelectContact`, formerly `ui.jsx:472`)
- Change: contacts tab shows accepted friends with presence dot and DM unread badge; click opens `GET /dms/:partnerId` and uses the same `MessageArea` / `Composer` (send goes to `POST /dms/:partnerId`). Friends panel: add by exact username (`GET /users?username=` then `POST /friends`), incoming requests with Accept/Decline, remove with confirm. Opening a DM calls `PATCH /dms/:partnerId/read`. Clicking a username/avatar opens the profile dialog (step 3.8).
- Acceptance: A adds B by username; B sees the request, accepts; both open a DM; A types → B sees the typing indicator; A sends → arrives live with no reconnect; B's badge shows 1 until the DM is opened.
- Effort: 5 h

Step 3.7 — Presence, typing, roster

- Closes: **P1-12** (client half), F-08 (typing/VOICE handlers)
- Files: `frontend/src/components/Roster.tsx`, `MessageArea.tsx`, `frontend/src/state/presence.ts` (new)
- Change: presence map `userId → status` seeded from `READY.online` and updated by `PRESENCE_UPDATE`; roster and contact rows read from it (offline if absent). `TYPING_START` shows "<name> is typing..." for 6 s per `(channel,user)`, cleared on that user's next message. Send op 3 `idle` on `visibilitychange` hidden, `online` when visible.
- Acceptance: close browser B's tab → within ~1 s of the socket closing A's roster shows B offline; reopen → online. Typing indicator appears and disappears.
- Effort: 3 h

Step 3.8 — Profile view and edit

- Closes: **P1-13** (profile edit), subject "A profile system (view user information)"
- Files: `frontend/src/components/ProfileDialog.tsx`, `SettingsDialog.tsx`
- Change: view dialog from `GET /users/:id`; own profile edit (`PATCH /me`: `roleTitle`, `currentActivity`, `avatarUrl` as a URL). If the team's user-management module owns avatar upload, link to it instead of duplicating.
- Acceptance: edit own `currentActivity` → refresh → persists; clicking another user's name shows their profile; no edit controls on others.
- Effort: 2 h

Step 3.9 — Responsive, accessible, console clean

- Closes: subject III.2 (console), III.3 (responsive/accessible)
- Files: `frontend/src/styles.css`, all components
- Change: every icon button gets `aria-label`; dialogs trap focus and close on Esc; message list has `aria-live="polite"`; layout works at 375 px (single column, sidebar as drawer). Run Chrome DevTools console on every flow of the demo script (step 5.4).
- Acceptance: Lighthouse accessibility ≥ 90 on the main screen; console empty through the whole demo script; phone-width screenshot shows a usable layout.
- Effort: 2 h

PHASE 4 — Voice (about 20 h)

Design in one paragraph: WebRTC **mesh** (every participant has one `RTCPeerConnection` per other participant), audio only, cap `VOICE_MAX_PEERS=6`. The signaling (offer/answer/ICE) is relayed by the gateway on the sockets that are already authenticated. The **joiner always creates the offers** to everyone already in the channel; that rule removes offer glare, so no "perfect negotiation" code is needed. Media never touches the server. Without TURN it works on the same network and most home networks; **TURN is required** when both peers are behind symmetric NAT or on a restrictive school/corporate network.

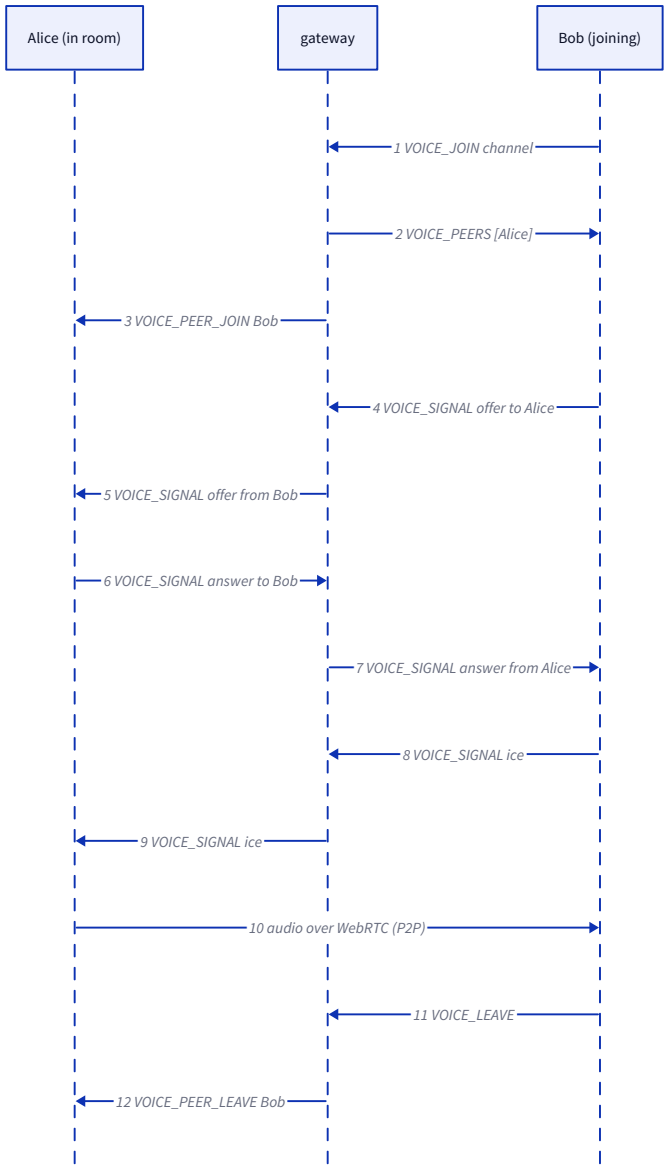


Figure 3 Voice flow

Step 4.1 — Gateway: voice state and relay

- Closes: **P1-14** (B-06, R-07), replaces `apps/voice-signaling`
- Files: `apps/gateway/src/app/protocol.ts` (ops 20–23, events), `apps/gateway/src/app/voice.service.ts` (new), `apps/gateway/src/app/gateway.ts:117-129` (switch), `apps/gateway/src/app/events.consumer.ts:149-164` (delete `onVoice`), `apps/gateway/src/app/config.ts` (remove `TOPICS.voice`), `apps/gateway/src/gateway.module.ts`
- Change:
- `VoiceService` holds `rooms: Map<channelId, Map<userId, {session, muted}>>` and `userRoom: Map<userId, channelId>`.
- `VOICE_JOIN {channel_id}`: check the channel is in `session.channels` **and** is type `VOICE` (look up once with Prisma, cache the type in a `Map`); if the user is already in another voice room, or in the same one from another socket, remove the old entry first (and send that old socket `VOICE_ERROR {code:'moved'}`). If the room has `VOICE_MAX_PEERS` people → `VOICE_ERROR {code:'full'}`. Else reply `VOICE_PEERS {channel_id, peers:[{user_id,username,muted}]}` to the joiner, send `VOICE_PEER_JOIN` to the others, and `VOICE_STATE_UPDATE {channel_id, user_ids}` to the whole guild (`registry.toGuild`) so the sidebar shows who is in the room.

- `VOICE_SIGNAL {to, kind: 'offer'|'answer'|'ice', payload}`: relay only if sender and `to` are in the **same** voice room; forward `{from, kind, payload}` as event `VOICE_SIGNAL`. Reject `payload` larger than 8 KiB. Per-socket limit 200 signals per 10 s.
- `VOICE_LEAVE` and `handleDisconnect`: remove from the room, send `VOICE_PEER_LEAVE {user_id}` to the others, update the guild roster.
- `VOICE_MUTE {muted}` → update state and broadcast `VOICE_STATE_UPDATE`.
- Remove the REST `join-voice` / `leave-voice` routes (`app.controller.ts:149-157`) and `chat.service.ts:501-545`; `Member.voiceChannelId` was already dropped in 2.1.
- Acceptance: two `wscat` sessions with tokens for alice and bob. Alice `{"op":20,"d":{"channel_id":"<voiceId>"}}` → `VOICE_PEERS` with no peers. Bob joins → Bob gets `VOICE_PEERS [alice]`, Alice gets `VOICE_PEER_JOIN bob`. Bob `{"op":22,"d":{"to":"1","kind":"offer","payload":{"sdp":"x"}}}` → Alice receives `VOICE_SIGNAL` with `from:"2"`. A third user (not in the room) sending `op:22` to alice → nothing delivered. Closing bob's socket → alice gets `VOICE_PEER_LEAVE`. A 7th join → `VOICE_ERROR full`.
- Effort: 4 h

Step 4.2 — ICE servers endpoint and optional coturn

- Closes: part of **P1-14**
- Files: `apps/chatsystem/src/voice.controller.ts` (new), `docker-compose.yml`, `infra/coturn/turnserver.conf` (new)
- Change:
- `GET /api/voice/ice` (authenticated) returns `{ iceServers: [{urls: STUN_URLS.split(',')}, …(TURN_URL ? [{urls: TURN_URL, username: TURN_USER, credential: TURN_PASS}] : [])] }`. The TURN secret never ships inside the JS bundle.
- compose service `coturn` under `profiles: ["turn"]` (image `coturn/coturn`, `network_mode: host` is the least painful on Linux, or publish `3478/udp,tcp` and a relay port range `49160-49200/udp`). `turnserver.conf`: `realm=chat`, `lt-cred-mech`, `user=<TURN_USER>:<TURN_PASS>`, `no-multicast-peers`, `no-cli`, `fingerprint`. Start with `docker compose --profile turn up -d`.
- Static long-term credentials are fine for an MVP; mention "time-limited HMAC credentials" in the README as the upgrade.
- Acceptance: `curl -sk https://localhost:8443/api/voice/ice -H "Authorization: Bearer $T" | jq .iceServers` shows the STUN entry (and TURN if set); without a token 401.
- Effort: 2 h

Step 4.3 — React voice hook (the core)

- Closes: **P1-14** (UI/media half)
- Files: `frontend/src/voice/useVoice.ts` (new), `frontend/src/voice/peer.ts` (new), `frontend/src/realtime/gatewayClient.ts` (`sendVoice*` helpers)
- Change (per peer, kept in a `Map<userId, Peer>`): ``ts async function join(channelId: string) { const { iceServers } = await chatApi.voiceIce(); stream = await navigator.mediaDevices.getUserMedia({ audio: { echoCancellation: true, noiseSuppression: true }, video: false }); gw.send({ op: 20, d: { channel_id: channelId } }); } // on VOICE_PEERS: for each existing peer -> createPeer(id, /*initiator*/ true) // on VOICE_PEER_JOIN: do nothing (the joiner will send us an offer) function createPeer(id: string, initiator: boolean) { const pc = new RTCPeerConnection({ iceServers }); stream.getTracks().forEach(t => pc.addTrack(t, stream)); pc.onicecandidate = e => e.candidate && gw.send({ op: 22, d: { to: id, kind: 'ice', payload: e.candidate } }); pc.ontrack = e => { audioEl(id).srcObject = e.streams[0]; }; // hidden <audio autoplay> pc.onconnectionstatechange = () => { if (['failed','closed'].includes(pc.connectionState)) removePeer(id); }; if (initiator) pc.createOffer().then(o => pc.setLocalDescription(o)).then(() => gw.send({ op: 22, d: { to: id, kind: 'offer', payload: pc.localDescription } })); peers.set(id, { pc, pending: [], remoteSet: false }); } // on VOICE_SIGNAL {from, kind, payload}: // offer : createPeer(from,false) if absent; await pc.setRemoteDescription(payload); flush pending ICE; // answer = await pc.createAnswer(); await pc.setLocalDescription(answer); send kind:'answer' // answer: await pc.setRemoteDescription(payload); flush pending // ice : peer.remoteSet ? pc.addIceCandidate(payload) : peer.pending.push(payload) // QUEUE until remote description ``
- `mute()`: `stream.getAudioTracks().forEach(t => t.enabled = !t.enabled)` and send op 23.
- `leave()` and cleanup: send op 21, `pc.close()` each peer, remove each `<audio>`, `stream.getTracks().forEach(t => t.stop())` (this also turns off the browser's mic indicator), clear the map. Call it on component unmount, on `pagehide`, and when the gateway socket closes or reports `VOICE_ERROR`.
- `VOICE_PEER_LEAVE` → `removePeer(id)`. A `VOICE_ERROR full|moved` → show a message and run the cleanup.
- Errors: `NotAllowedError` (mic denied) and `NotFoundError` (no mic) show a readable message and never send `VOICE_JOIN`.
- Acceptance: two Chrome windows (or one + a second profile) on `https://localhost:8443`, both join the same voice room, grant the mic → each hears the other (test with a phone playing audio or use Chrome's `--use-fake-device-for-media-stream` flag to show non-silent tracks; verify `pc.getStats()` inbound-rtp `bytesReceived` increases). Leaving in one window removes its `<audio>` in the other and the mic indicator turns off.
- Effort: 7 h

Step 4.4 — Voice UI

- Closes: **P1-14** (UI)
- Files: `frontend/src/components/VoiceBar.tsx` (new), `ChannelPanel.tsx` (voice rooms listed with participant avatars from `VOICE_STATE_UPDATE`), `StageHeader`
- Change: voice rooms (`type VOICE`) show a speaker icon and the live participant list; click → join. A bottom bar while connected: room name, mute toggle, leave button, connection state per peer (connecting/connected/failed). A "speaking" ring is optional (skip for the MVP).
- Acceptance: three windows join; the roster under the voice room shows all three; mute shows the muted icon in the other windows; leave removes the participant everywhere; the 7th user sees "Room full".
- Effort: 3 h

Step 4.5 — Voice test matrix (including a second device over HTTPS)

- Closes: voice acceptance; traps verified
- Change: test 2 and 4 participants on one machine; test from a phone/laptop on the same LAN using `https://<host-ip>:8443` (accept the cert warning). `getUserMedia` only works on a **secure context**: `localhost` or HTTPS. Plain `http://<ip>` will fail with `NotAllowedError` / `undefined mediaDevices`. Try mobile data vs Wi-Fi: if media does not flow, start coturn, set `TURN_*` in `.env`, retest. Record what works in the README "Known limits".
- Acceptance: the matrix table (device pair × network × result) is filled in the README; at least "two browsers, same machine" and "two machines, same LAN" pass.
- Effort: 3 h

PHASE 5 — Tests and delivery (about 16 h)

Step 5.1 — The 5 key tests

- Closes: **P2-2** (R-04, B-39), REPORT §6
- Files: `apps/chatsystem/src/auth/auth.guard.spec.ts`, `apps/chatsystem/src/chat.service.spec.ts`, `apps/gateway/src/app/events.consumer.spec.ts`, `apps/gateway/src/app/voice.service.spec.ts` (6th, short), `jest.config.ts` (fix, drop `--experimental-vm-modules` if the setup is CJS), delete the 9 boilerplate specs
- Change (Prisma mocked with a plain object; Kafka mocked):
- **Auth guard**: no token → 401; valid token + body `senderId:"3"` → message stored under the token's user.
- **Membership**: non-member cannot post to or read a guild room; member can.
- **Edit/delete ownership**: another user's edit/delete → 403; empty/too long edit → 400.
- **DM access**: a non-party cannot read a DM; DM without friendship → 403.
- **Gateway delivery**: `onMessage` sends only to sessions indexed on that channel; `MESSAGE_EDIT` becomes `MESSAGE_UPDATE`; identify with a bad token closes with 4004.
- (small) voice relay refuses a signal to someone in another room.
- Acceptance: `npm test` → all suites pass and the count is not 9 boilerplate tests; `npm test -- --coverage` shows the guard, service and consumer covered.
- Effort: 6 h

Step 5.2 — E2E smoke script

- Closes: REPORT §6 "keep the live smoke"
- Files: `scripts/smoke.sh` (new) or `scripts/smoke.ts` using `ws`
- Change: against a running stack: dev-token ×2 → alice posts in room 1 → bob's WS receives `MESSAGE_CREATE` within 2 s → alice edits (bob gets UPDATE) → alice deletes (bob gets DELETE) → friend request/accept → DM delivered live → bob WS joins a new room created after connect (membership refresh) → exit non-zero on any failure. Optionally include the Kafka-restart case from step 2.3.
- Acceptance: `./scripts/smoke.sh https://localhost:8443` prints `OK 7/7` and exits 0; stop the gateway → it fails with a message naming the step.
- Effort: 3 h

Step 5.3 — README (chat part)

- Closes: **P2-10** (B-38), subject chapter VI (chat contribution)
- Files: `README.md`
- Change: replace the Nest template. Sections the subject requires that touch chat: description, instructions (`cp .env.example .env && docker compose up --build`, URL, dev tokens, `--profile turn`), technical stack with justifications (React, NestJS, Prisma/Postgres, Kafka, Redis, WebRTC), **database schema** diagram (generate with `prisma-erd-generator` or draw by hand), features list with the owner's name, modules claimed with points, known limits (one gateway instance, mesh voice ≤ 6, TURN needed on strict networks, self-signed cert). The first line must follow the subject's exact italic line, which is the team's to write once.
- Acceptance: a teammate who never saw the repo gets the stack running from the README alone.
- Effort: 2 h

Step 5.4 — Demo script and evaluation checklist

- Closes: subject Chapter VIII (evaluation, "demonstrate each claimed module")
- Files: `docs/DEMO.md` (new)
- Change: a 5-minute script with exact clicks: (1) one command up; (2) login via the team's auth (or dev login) in two browsers; (3) create collective, room, invite; join by invite; (4) messages live both ways, edit, delete; (5) friend request → accept → DM live, typing, unread; (6) presence on close/reopen; (7) kill the gateway container → banner → recovery; (8) voice with two users, mute, leave; (9) Privacy/ToS pages; (10) show `.gitignore` / `.env.example`. Add the checklist table mapping each module to the demo step proving it.
- Acceptance: someone who is not the author runs the script in ≤ 7 minutes without improvising.
- Effort: 2 h

Step 5.5 — Final clean pass

- Files: whole repo
- Change: `npm run lint`, `npm test`, `./scripts/smoke.sh`, clean `docker compose down -v && up --build`, Chrome console check on the demo path, `git ls-files` review (no secrets), small commits with clear messages (subject III.2 requires meaningful

commits from every member).
- Acceptance: all green on a clean clone.
- Effort: 3 h

---

8. Traps to avoid (specific to this stack)

1. **BigInt serialization.** Prisma returns `BigInt`; `res.json()` throws "Do not know how to serialize a BigInt". Shape ids to strings in every response (keep `shapeMessage`) and add the `BigInt.prototype.toJSON` safety net. On the client never use `Number(id)`: ids can exceed 2^53 and silently round. Compare with `BigInt(a) < BigInt(b)`.
2. **`BigInt(input)` throws on junk.** `BigInt('abc')` is a 500. Always go through `ParseBigIntPipe` (step 2.2), also for ids inside DTO bodies and WS frames (`ID_RE`).
3. **Ordering by `message_id`, never by arrival time or string compare.** `"9" > "10"` as strings. Kafka orders per partition key (`channel:<id>`), not across channels, and a reconnect replay can interleave: sort on the client by BigInt id and dedupe by id.
4. **Kafka consumer group per gateway instance.** The gateway uses `gateway-<uuid>` (`kafka.service.ts:182`) so every instance sees every event, which is right. Consequence: each restart leaves a dead group in Kafka (harmless for an MVP). Do not "fix" it with a fixed group id and then run two instances (they would split events). With `fromBeginning:false` events published while the gateway is down are lost; the client refetch on reconnect covers it.
5. **Kafka advertised listeners.** `localhost:9092` works from the host and breaks from containers. Keep two listeners (step 0.5). Symptom: "Connection error: ECONNREFUSED 127.0.0.1:9092" inside the container.
6. **Kafka producer that never reconnects.** The current `connected=false` is permanent (`kafka.service.ts:22-30`). Use the lazy connect + retry in step 2.3 and never `await` it in the HTTP path.
7. **Single gateway instance assumptions.** Voice state, presence offline detection and the replay buffer are in the gateway's memory/Redis keyed per instance. Do not scale `gateway` to 2 replicas in the MVP. Write it in the README.
8. **React StrictMode double socket.** In dev, effects run twice: you get two WebSockets and two IDENTIFYs. Connect in `useEffect` with a cleanup that disconnects, guard callbacks with `socket !== this.socket`, and test in dev, not only in the production build.
9. **WS auth on reconnect.** The token must be read again on every reconnect (it may have been refreshed or cleared). Never send it in the URL query string (proxies log it). Do not auto-retry on 4004 or 4011 or you spam the gateway with a dead token forever.
10. **`setTimeout` overflow.** For the expiry timer, clamp to `2**31-1` ms; a larger delay fires immediately and closes every session at once. Also reject tokens whose `exp` is in the past.
11. **JWT pitfalls.** Always pass `algorithms:['HS256']`; require `exp`; treat `sub` as an integer string and validate it; do not trust `username` for authorization, only for display; boot must fail without `JWT_SECRET`.
12. **CORS with credentials.** There are no cookies in this design (Bearer header), so `credentials` stays off and you may use an origin allow-list instead of `origin:true`. The one-origin proxy removes CORS in production. If someone later moves the token to a cookie, `Access-Control-Allow-Origin:*` stops working with credentials and WS needs an `Origin` check.
13. **WS frame limit vs SDP.** A tight `maxPayload` breaks voice: audio offers are 2–4 KB. Use 16 KiB and a per-frame check for `VOICE_SIGNAL` (8 KiB).
14. **Reverse proxy must upgrade WS.** Without `Upgrade` / `Connection` headers and `proxy_http_version 1.1` the gateway returns 400/426 behind nginx. Set `proxy_read_timeout` above the heartbeat interval (41 s).
15. **Migrations squash.** Squashing is safe only because no real data exists. Delete the volume (`docker compose down -v`) after squashing or `migrate deploy` will complain about history mismatch. Never edit a migration after another teammate applied it.
16. **Two prisma clients.** `generated/prisma` must be generated at install/build (`postinstall: prisma generate`), not committed, and both apps import the same output path; Docker's build stage runs `prisma generate` before `nest build`.
17. **Voice: ICE candidates arrive before the remote description.** Queue them per peer and flush after `setRemoteDescription`; otherwise `addIceCandidate` throws and the call silently fails one time in five.
18. **Voice: glare.** If both sides send offers you get "InvalidStateError". The joiner-always-offers rule prevents it; do not add "renegotiate on track change" in the MVP.
19. **Voice: cleanup.** Close every `RTCPeerConnection`, stop every track (mic light stays on otherwise), remove `<audio>` elements, handle tab close (`pagehide`), socket close and `VOICE_ERROR`. Leaks show up as ghost participants and a second join hearing echo.
20. **Voice: HTTPS.** `navigator.mediaDevices` is `undefined` on plain HTTP off localhost. Evaluators will often open the app through an IP or hostname: the HTTPS proxy is mandatory, not cosmetic. Also, browsers require a user gesture before playing audio: start playback from the Join button click.
21. **Voice: NAT.** STUN alone fails across strict NATs (school networks). If the demo network is unknown, bring coturn up and set `TURN_*` before the evaluation.
22. **Upsert races.** Guard and WS identify both upsert the same user row: use `upsert` (not find-then-create), and ignore a `P2002` retry once.
23. **Presence offline must carry `guild_ids`.** The session is removed before `offline` is published, so the consumer can no longer look up the user's guilds (step 2.8).
24. **Do not leak secrets into the frontend bundle.** Only `VITE_*` are bundled; TURN credentials come from `/api/voice/ice`, `JWT_SECRET` is never in a `VITE_` variable.
25. **Dev-token endpoint in production.** It must not exist when `NODE_ENV=production`. The compose file for the final demo should set it, and a test asserts `POST /api/dev/token` → 404 in that mode.

---

9. Contract for the auth teammate (read this alone)

You own sign-up, login (local + OAuth), password hashing and the `user` data. The chat service does **not** build login. It needs only this:

9.1 The token you issue

- Format: **JWT, algorithm HS256**.
- Secret: env var `JWT_SECRET`, at least 32 random characters, the **same value** in the `.env` of the auth service, `chatsystem` and `gateway`. Never committed. (If the team prefers RS256 later, the chat verify function is one function; say so early.)
- Required claims:

claim	type	meaning
`sub`	string of a **positive integer** (max 18 digits)	the user id. **Must be an integer.** If your ids are UUIDs, tell the chat owner on day 1
`username`	string, 3–32 chars	unique, display name
`exp`	number	expiry. Please **1 hour** from issue. Required
`iat`	number	issued-at
`avatar`	string URL, optional	shown as avatar

- `sub` never changes for a user. `username` may change; chat updates its copy on the next token.
- Do **not** put the email, roles or secrets in the token. Chat does not need them.

9.2 How the token reaches the chat

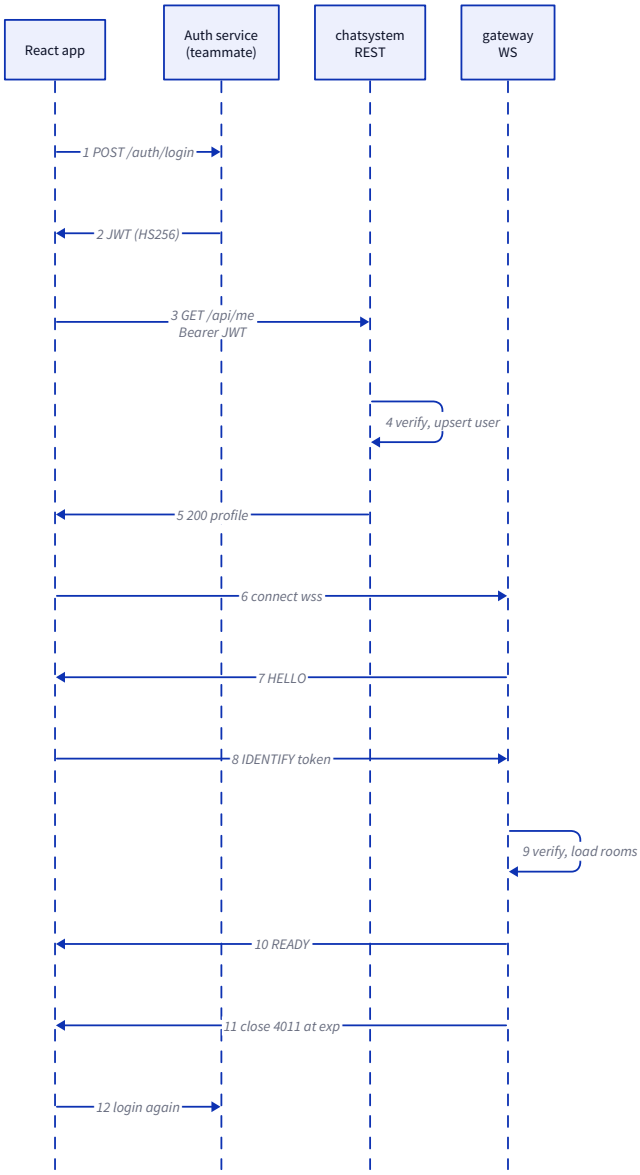


Figure 4 Login, token and identify flow

1. All three things (React chat app, chat API, auth service) are reached through **one HTTPS origin**, e.g. `https://localhost:8443`. Auth is exposed at `/auth/*`. Chat is at `/api/*` and `/gateway`.
2. After a successful login (local form **or** OAuth callback), your page stores the token in `localStorage['access_token']` and redirects to `/` (or to `?next=`). Chat reads that key.
3. The chat sends it as `Authorization: Bearer <jwt>` on every REST call and as `d.token` in the WebSocket `IDENTIFY` frame.
4. Logout: remove `localStorage['access_token']`. Chat listens for that and logs out.
5. Your login page must live at `/login` and accept `?next=<path>`. Chat redirects there on a 401 or when the WebSocket is closed with code 4004 or 4011.
6. Optional but welcome: `POST /auth/refresh` (with a still-valid token) returning `{ "token": "<new jwt>" }`. If present, chat tries it before redirecting to login.

9.3 What chat does with the token

- Verifies signature (`HS256` only), `exp`, and `sub` format. Invalid → REST 401, WS close 4004. Expired during an open socket → WS close **4011**.
- Creates/updates its own profile row `user(user_id = sub, username)` on first use. Chat never reads your database and never stores credentials.
- Gateway closes the socket at the token's `exp`.

9.4 What we need from you, with dates

What	When
Agree: HS256 + `JWT_SECRET`, integer `sub`, `/auth/*`, `/login` route	day 1

What	When
`POST /auth/login` returning a token matching 9.1, even if only for 2 seeded users	day 4
Two real test users in your seed (ids 1 and 2 are fine)	day 4
OAuth flow ending with the same `localStorage` write	before the final integration
Your service in the shared compose + a `location /auth/` block in the proxy	day 4

9.5 Until then

Chat ships a **dev-only** `POST /api/dev/token {userId, username}` (returns a valid token) and a dev login form, both disabled when `NODE_ENV=production`. Chat work is not blocked by you.

---

10. Team split and interface

The team is 4 people. The chat owner (the friend) owns **only** the chat system. Here is exactly what chat needs from the others and what chat gives back.

Need	From whom	Why / default if nobody provides it
Login + JWT per section 9	auth teammate	dev token covers development, real login needed for the evaluation
One shared `docker compose` and one HTTPS proxy	devops / whoever owns the repo root	chat provides `docker-compose.yml` services and the three nginx `location` blocks (step 0.6). If nobody owns this, chat's compose with its `proxy` profile is the shared one
Shared Postgres instance	devops	chat creates and migrates its **own database** (`chat`); auth uses another. Never one shared `user` table
Frontend shell integration (if the team has one app)	frontend/shell owner	agree one of: (a) chat SPA served at `/chat/` with `VITE_BASE=/chat/`, same origin, same `localStorage` token (recommended); (b) later, move components into the shell. Navigation, logo, theme belong to the shell
Privacy Policy and Terms of Service	whoever owns the shell	mandatory for the whole project; chat links to them and adds its own data paragraph (messages, friends, presence, voice is not recorded)
Friends API ownership	agree with the User Management owner	this plan makes chat own friends and online status; the user-management module must not build a second one
Game invites in chat	game owner (optional)	chat exposes nothing for it in the MVP

What chat gives the team: `/api/*`, `/gateway`, `/api/voice/ice`, a static React build, a compose file, the section 9 contract, the smoke script.

Joint integration test (auth teammate + chat owner, day 5, again at the end):

1. `docker compose up --build` from a clean clone, using the real auth service.
2. Register user A (local) and user B (OAuth if available) in two browsers.
3. A creates a collective and invites B; B joins by invite.
4. A and B exchange messages, edit, delete; befriend; DM; typing; presence.
5. Both join a voice room.
6. Corrupt the token (edit one char in `localStorage`) → both REST and WS reject it and land on `/login`.
7. Wait for token expiry (use a 60 s token in a test build) → the socket closes with 4011, the app asks to log in, after login messages resume.
8. Run `./scripts/smoke.sh` with two real tokens.

Each person demonstrates their own parts in the evaluation. The chat owner must be able to explain the gateway, Kafka path, JWT verification and the WebRTC mesh.

---

11. Timeline (one developer; auth teammate in parallel)

Day	Chat developer	Auth teammate (parallel)
1	0.1, 0.2, 0.3, 0.4 (cleanup, env, Dockerfiles). Send section 9 to the team; agree on the proxy owner	read the contract; decide JWT library; `sub` integer
2	0.5, 0.6, 0.7 (compose, HTTPS proxy, lint)	service skeleton + DB
3	1.1, 1.2, 1.3, 1.4 (auth guard, spoof fixes, gateway verify, throttler)	`POST /auth/login` issuing the contract token
4	2.1, 2.2 (schema squash, validation)	login + seed; **drop into compose and proxy**
5	2.3, 2.4, 2.5 (Kafka retry, edit/delete events, DMs). **First joint integration test** (section 10, steps 1–4)	OAuth flow
6	2.6, 2.7, 2.8 (friends, membership refresh, presence)	OAuth, refresh endpoint
7	2.9, 2.10 + 3.1 (invites, unread, React hygiene)	`/login` page ready, Privacy/ToS if they own it
8	3.2, 3.3, 3.4 (token, gateway client, message list)	—
9	3.4 end, 3.5, 3.6 (collectives/rooms/invites UI, DMs/friends UI)	—
10	3.7, 3.8, 3.9 (presence/typing, profile, responsive/a11y)	—
11	4.1, 4.2 (gateway voice, ICE endpoint, coturn)	—
12	4.3 (React voice hook)	—
13	4.4, 4.5 (voice UI, test matrix incl. second device). **Second joint integration test** (full, section 10)	participates in the test
14	5.1, 5.2, 5.3, 5.4, 5.5 (tests, smoke, README, demo, final pass)	reviews the demo script

Day total 14 working days ≈ 111 h of steps (0: 11, 1: 9, 2: 24.5, 3: 32, 4: 19, 5: 16). Buffer: keep 2 days free on top; voice and the React rewrite are the items most likely to slip. If you must cut, cut in this order: step 3.8 (profile edit), the typing indicator, DM unread, voice for 5–6 peers (keep 4), never auth, HTTPS, or the Chrome console cleanliness.

---

12. Coverage table: every audit TODO → blueprint step

Audit ID	Title (short)	Blueprint step
P0-1	Real authentication end to end	1.1, 1.2, 1.3, 3.2, section 9 (login/hash are the teammate's, not chat's)
P0-2	DM snooping and forced friendships	1.2, 2.5, 2.6
P0-3	Open "join any guild" endpoint	2.9
P0-4	Committed `.env`, ports on all interfaces	0.1, 0.3, 0.5
P0-5	WS and REST abuse limits	1.3 (payload, per-user cap, stats), 1.4 (throttler), 0.6 (proxy rate limit, optional)
P1-1	DMs work live	2.5, 3.6
P1-2	Membership refresh after open	2.7
P1-3	Edit and delete events + UI	2.4, 3.4
P1-4	Filter messages by open room	3.4
P1-5	React env config	3.1
P1-6	Not-silently-lossy delivery (Kafka)	2.3 (producer fix), 3.3 (refetch on reconnect)
P1-7	400/404 instead of 500	2.2
P1-8	Scope reads to the caller	2.2 (CORS/origin), 2.6 (`GET users` list removed), 2.9 (room previews, profile fields), 1.3 (WS cors removed)
P1-9	React reconnect, resync, errors	3.2 (401), 3.3
P1-10	Room previews "Unknown"	2.10
P1-11	Unread badges	2.10 (DMs persisted), 3.4 (rooms client-side)
P1-12	One presence source	2.8, 3.7
P1-13	Migration gaps (create guild/room, invites, profile, edit/delete, DM, friends)	3.5, 3.6, 3.8, 3.4, 2.6
P1-14	Voice: remove fake huddle / build real	3.1 (remove fake), 4.1–4.5 (real voice)
P1-15	Untrack build output and deps	0.1
P2-1	Real lint	0.7
P2-2	Replace boilerplate tests	5.1
P2-3	Schema/migration drift and names	2.1
P2-4	Consistent `.env` ports	0.3
P2-5	Delete dead code and deps	0.2
P2-6	Clean the React package	3.1
P2-7	Small data races	2.5 (unique `dmKey`, upsert, no DM-on-list), 3.4 (client sort)
P2-8	Invite expiry and revoke	2.9
P2-9	Remove fake UI	3.1
P2-10	Real README	5.3
P2-11	Pin toolchain, gateway prod start	0.4, 0.7
P2-12	Startup `TimeoutNegativeWarning`	0.7 (30 min timebox; if not found, listed in README known issues. **Deferred after MVP** unless it appears in the console, since it is a *suspected* Node warning, not user-visible)
P2-13	Unused schema columns, no leave/kick	2.1 (drop unused columns). Leave/kick/delete guild flows: **deferred after MVP**, not required by the subject's chat bullets

Nothing else from the audit is dropped. Items noted as "Not a TODO" in the audit (B-31 SQL injection, F-10/F-11 XSS) need no action.

Appendix — Optional simplifications after delivery

Not part of the MVP; do these only if time remains and the evaluation is done.

1. Replace Kafka with an HTTP call or Redis pub/sub from `chatsystem` to `gateway`, keep the two services (microservices module stays valid).
2. Replace the hand-rolled registry/replay buffer in the gateway by Socket.IO rooms, or delete the replay buffer and ack code entirely (the client already refetches on reconnect).
3. Move presence to a `Map` in the gateway and drop Redis if a single gateway is the only target.
4. Voice: swap the mesh for an SFU (mediasoup/LiveKit) if more than 6 participants are ever needed; time-limited HMAC TURN credentials.
5. Add block-user, read receipts and message search (the subject's "Advanced chat features" and "Advanced search" modules).
6. Merge `chatsystem` and `gateway` into one Nest app (same REST + WS) if the microservices module is not claimed.

Part B — ft_transcendence Subject Compliance

Subject version 21.2. Repo: `/home/mushigarou/AI_Lab/chatsystem` at `e62e910e`. Scope: the chat system only.

Method: I read the whole subject and `REPORT.md`, then re-checked status claims against the repo. Everything marked "runtime" comes from the earlier audit run. I did not re-run it, and the React app was never clicked through in a browser. Items I could not verify are marked Unknown or "not browser-tested".

Summary

	Points
Credible NOW (fully functional only)	**about 3** (Framework major 2 + ORM minor 1). Both depend on the team's app being React + NestJS and running cleanly in Chrome, which nobody has checked.
After the MVP blueprint (P0–P5)	**about 7–8 chat-owned**: WebSockets real-time 2 + basic chat/profile/friends 2 + frameworks 2 + ORM 1, plus voice as a Module of choice (1) and microservices (2, shared). Stretch: advanced chat 1, organization system 2.
Needed	14 for the team, from the 4 people together. The chat owner cannot reach 14 alone.

Biggest risks

1. **Rejection risk on the mandatory part.** The subject says "If you don't follow them, your project will be rejected." Right now the chat repo has no Dockerfiles and no single-command deployment. It has no HTTPS, and no Privacy Policy or Terms of Service. `.env` is committed and `.gitignore` is broken. There is no `.env.example`, and the README is the unmodified Nest template. Several of these are team-level, but the chat repo fails them today.
2. **90% done = 0 points.** The subject says "Non-functional or incomplete modules = 0 points." These are close but still score zero:
3. Real-time: DMs never arrive live, edit/delete arrive as new messages, and membership is frozen at connect.
4. Basic chat: there is no remove-friend, no profile UI, and no way to create a room or guild in React.
5. Voice: it is currently a fake button.
6. **No authentication.** Identity is a number the client supplies. The evaluator's first move, spoofing another user, succeeds with one `curl`. This also blocks "Standard user management", which belongs to the auth teammate.
7. **Dependency on teammates.** Auth (JWT), HTTPS or a reverse proxy, and one shared compose file all come from others. Without them none of the chat modules can be demonstrated properly.
8. **Advanced chat.** Two of its six bullets need a game, and nobody has said a game exists.

1. Mandatory requirements matrix

Legend: Chat-owned = the chat owner must deliver it. Team-level = one team deliverable. Status: Met / Partial / Missing / Unknown (team).

General requirements (III.2) and preamble

Requirement (quoted short)	Applies	Status	Evidence	What to do
"web application, and requires a frontend, backend, and a database"	Team-level; chat supplies all three	Partial	React (Vite), NestJS (`apps/chatsystem`, `apps/gateway`), Postgres (`prisma/schema.prisma`). Builds and boots (audit).	Merge into the team app, or make sure the chat is reachable from the team's front end.
"Git ... clear and meaningful commit messages ... Commits from all team members ... Proper work distribution"	Team-level	Unknown (team)	In this repo: 10 commits by "Jassim Bahmida", 6 by "futex" and 3 by "qwen.ai[bot]". Messages are weak ("fix all the problems in the front, but the design is not complet").	Commit under your own identity. Use meaningful messages. Show your own commits in the team repo. Remove the bot as a co-author.
"Deployment must use a containerization solution ... and run with a single command"	Team-level; chat-owned part is the apps	**Missing**	No `Dockerfile*` anywhere (`find` of the working tree, 0 hits). Only `infra/postgres/docker-compose.yml` exists, with postgres, redis and kafka (lines 2, 21, 33). The apps run from host Node.	P0: write a Dockerfile for each app (chatsystem, gateway, and the voice signaling if kept). Add them and the front end to one compose file. `docker compose up --build` must start everything.
"compatible with the latest stable version of Google Chrome"	Chat-owned (front end)	Unknown	React app was built (`vite build` passes) but never opened in a browser.	P3/P5: manual pass in Chrome.
"No warnings or errors about the Javascript code ... in the browser console"	Chat-owned (front end)	Unknown	Not tested. Likely issues: unguarded `JSON.parse` (`gatewayClient.ts:61`), failed requests while the backend is down, the CORS flood from a different origin.	P5: open DevTools on every flow and fix everything, including React warnings. Dependencies are `latest`, so pin them.
"accessible Privacy Policy and Terms of Service pages with relevant content"	Team-level (one page set)	**Missing**	No such page or file in the repo (`git ls-files` outside `node_modules` shows none).	Add `/privacy` and `/terms` pages and footer links, with real content. Describe the data the chat stores (messages, presence, usernames). The auth teammate should state what they store too. Coordinate so there is one page. Missing pages are listed as a rejection reason.
"Multi-user Support ... simultaneously ... No data corruption or race conditions"	Chat-owned	Partial	Live delivery to a second client works (audit step 7e). Known races: `getOrCreateDm` check-then-create with no transaction (`chat.service.ts:461-478`). Presence is split between DB and Redis.	P2: transaction plus unique-violation handling. One source of truth for presence. Test with two browsers.
"Real-time updates are reflected across all connected users when applicable"	Chat-owned	Partial	Works for guild channels only. DMs not live (`session-loader.service.ts:54-58`). New channels or guilds get nothing until reconnect. Edits and deletes show up as new messages (`events.consumer.ts:19-31`).	P2. See the real-time module below.
"Team roles ... PO, PM, Tech Lead, Developers ... documented in README.md"	Team-level	Unknown (team)	README is the Nest template.	Team README. Four people can combine roles. The chat owner supplies a short "role + responsibilities + contributions" entry.

Technical requirements (III.3)

Requirement	Applies	Status	Evidence	What to do
"A frontend that is clear, responsive, and accessible across all devices"	Chat-owned (chat UI)	Partial	Custom `styles.css`, one monolith `ui.jsx` (539 lines). A few `aria-label`s, tabs with `role`. Not tested on small screens. Fake UI (Copilot, search, notifications) is present.	P3: test on a phone-width viewport. Remove fake controls. Fix labels and keyboard use.
"A CSS framework or styling solution of your choice"	Chat-owned	Met	Plain CSS in `front-end/react replacement/src/styles.css`. Custom CSS counts as a "styling solution".	Mention it in the README.
"credentials ... in a local .env file that is ignored by Git, and provide an .env.example file"	Team-level; chat repo fails	**Missing**	`.env` is tracked (`git ls-files`). `.gitignore` contains the text `(empty)` and ignores nothing. The only example is `infra/postgres/env.example`, which is not a root `.env.example`. The password is already in git history.	P0: `git rm --cached .env`, rotate the password, add a real `.gitignore`, add root `.env.example` with every variable (including the JWT secret).

Requirement	Applies	Status	Evidence	What to do
"The database must have a clear schema and well-defined relations"	Chat-owned	Partial	`prisma/schema.prisma` has User, Guild, Channel, Member, Message, Friendship, Invite, DirectChannel, DirectChannelMember. Relations exist with cascades. `Role` is unused. Two FKs on `message.channel_id` drifted from the migration (`schema.prisma:111-112` vs migration `20261009053621_init`). No user email or password column.	P2: squash migrations, fix the drift. Add the auth columns together with the teammate. Drop unused tables. Document the schema in the README.
"basic user management ... sign up and log in securely: email and password ... hashed passwords, salted"	Auth teammate	Missing in this repo	`User` has no email or `passwordHash` (`schema.prisma:20-36`). `argon2` and `@nestjs/jwt` are installed but never imported.	Auth teammate's job. Chat consumes their JWT and uses the user id from the token (P1). Agree on the shared user table or service early.
"All forms and user inputs must be properly validated in both the frontend and backend"	Chat-owned	Partial	Backend: manual checks only (name trim, 4000-char limit in `createRoomMessage`). No `ValidationPipe` . Bad input gives 500 (`/users/abc` , empty body). `dto.ts` is imported by nothing. Frontend: little visible validation.	P2: DTOs plus a global `ValidationPipe` , 400 and 404 responses. Frontend limits (length, empty) on message, room name and profile forms.
"Any connection to the backend ... must use HTTPS"	Team-level (reverse proxy); chat must work behind it	**Missing**	Plain `http://localhost:3000` and `ws://localhost:3001/gateway` . Endpoints hardcoded in the React client (`gatewayClient.ts:10` , `chatApi.ts:3-5`).	P0: one TLS terminator in the team's compose (nginx or caddy). The chat uses `wss://` . Read URLs from `VITE_*` env, not hardcoded. Fix CORS and the `Origin` check for the new origin.
Framework definition (React and NestJS count)	Chat-owned	Met	React + Vite; NestJS 12.	None. Pin versions (they are `latest`).
Passwords hashed, XSS and SQL injection (security expectations)	Chat-owned partly	Partial	Prisma queries are parameterized (audit B-31). No XSS found in React or vanilla (audit F-10/F-11); React escapes text by default, and no `dangerouslySetInnerHTML` in `ui.jsx` . No auth, no password storage here.	Keep rendering messages as plain text. Add rate limits and a WebSocket `maxPayload` (P0-5).
Secrets not in repo	Chat-owned	**Missing**	`.env` tracked, DB password in history. Postgres, Redis and Kafka published on all interfaces.	P0 (see the `.env` row). Bind ports to `127.0.0.1` .
README requirements (VI)	Team-level	**Missing**	`README.md` is the Nest template.	Needs: first line in italics ("This project has been created as part of the 42 curriculum by ..."), Description, Instructions, Resources (including how AI was used), Team Information, Project Management, Technical Stack, Database Schema, Features List, Modules (points), Individual Contributions. In English. Supply your section with the chat parts.
"Only the work inside your repository will be evaluated" and "brief modification ... may be requested"	Team-level	Unknown	The chat must live in the team's repo.	Merge or submodule into the team repo. Be able to change a function live (e.g. message length rule, a new event) without breaking.
Repo hygiene (supports Git, `.env` and "commits" rules)	Chat-owned	**Missing**	34,566 `node_modules` files, `dist/` , generated Prisma client tracked. Stray files (`.backend.md.swp` , `some_test.*` , `tmp.md` , `files.zip`).	P0/P1: the untrack commands from the audit's section 7.

AI rule: the subject (chapter I) says you must be able to explain any AI-generated code during peer evaluation. The commit history shows a bot co-author, so be ready to explain every file, especially the gateway state code.

2. Module matrix

Points per the subject: Major = 2, Minor = 1. 14 points needed. Bonus counts only past 14, capped at 5.

Module (exact name)	Type, pts	Subject criteria (short)	Owner	Status now	Gap, blueprint phase	Dependencies
Web: "Major: Use a framework for both the frontend and backend."	Major, 2	Frontend framework (React...) and backend framework (NestJS...)	Shared (counts once for the team)	**Satisfied** for chat parts, provided Chrome runs cleanly	P5: Chrome pass. Pin versions. All teammates should use the same stack, or the evaluator sees a mixed picture.	None
Web: "Minor: Use a frontend framework" / "Minor: Use a backend framework"	Minor, 1 each	Same, separately	Shared	Alternative only	Do not claim these in addition to the Major above.	Replaced by the Major
Web: "Major: Implement real-time features using WebSockets or similar technology."	Major, 2	Real-time updates across clients; handle connection/disconnection gracefully; efficient message broadcasting	Chat	**Near**. Missing: DMs not live (B-02), edit/delete as creates (B-05), frozen membership (B-03), no room filter in UI (F-02), reconnect counter never reset (F-05), no resync on reconnect, no auth on WS	P2 (gateway fixes) and P3 (client handling)	None
Web: "Major: Allow users to interact with other users."	Major, 2	"basic chat system (send/receive messages between users)"; "profile system (view user information)"; "friends system (add/remove friends, see friends list)"	Chat (chat + friends), with the auth teammate (profile data)	**Near**. Send/receive works in guild rooms, not DMs; add-friend exists in API only (`app.controller.ts:190`); remove-friend does not exist; no React UI for DMs, friends or profile	P2 (remove-friend, DM fix) and P3 (UI: DM screen, friends list, profile view). Needs P1 so users are real.	Required by Advanced chat. Profile/friends data overlaps Standard user management.
Web: "Minor: Use an ORM for the database."	Minor, 1	ORM	Shared	**Satisfied** (Prisma, `prisma/schema.prisma`, migrations apply, seed runs)	Clean the migrations (P2).	None
Web: "Minor: A complete notification system for all creation, update, and deletion actions."	Minor, 1	Notifications for create/update/delete	Shared	Not started (a "Notifications" button is fake UI)	Not recommended	Needs wide coverage across everyone's features
Web: "Minor: File upload and management system."	Minor, 1	Multiple types, client and server validation, secure storage with access control, preview, progress, delete	Chat could own	Not started (attachments are turned into text, `ui.jsx:254`)	Stretch only; large for 1 point	Avatar upload belongs to user management
Web: "Minor: Implement advanced search functionality with filters, sorting, and pagination."	Minor, 1	Search + filters + sorting + pagination	Chat could own	Not started (topbar search is fake)	Stretch: message search with `before` pagination partly exists in the API (`app.controller.ts:86`)	None
Web: "Major: A public API to interact with the database with a secured API key, rate limiting, documentation, and at least 5 endpoints"	Major, 2	API key, rate limit, docs, 5+ endpoints	Not chat	Not started (routes exist but no key, no limit, no docs)	Do not claim	None
Web: "Minor: Real-time collaborative features" / "SSR" / "PWA" / "Custom-made design system"	Minor, 1 each	See subject	Other	Not started	Do not claim. A design system needs 10 reusable components; `ui.jsx` is a monolith.	None

Module (exact name)	Type, pts	Subject criteria (short)	Owner	Status now	Gap, blueprint phase	Dependencies
Accessibility: "Major: Complete accessibility compliance (WCAG 2.1 AA)..."	Major, 2	Screen reader, keyboard, assistive tech	Team	Not started	Do not claim	None
Accessibility: "Minor: Support for multiple languages (at least 3 languages)" and "Minor: Right-to-left (RTL) language support" and "Minor: Support for additional browsers"	Minor, 1 each	i18n with switcher; RTL layout mirroring; 2 extra browsers	Team	Not started (all text hardcoded, `lang="en"`)	Leave to the team unless someone owns the front end	None
User Management: "Major: Standard user management and authentication."	Major, 2	Update profile; upload avatar (default if none); add friends and see online status; profile page	Auth teammate (chat supplies friends and presence)	Not started in this repo (no auth, no avatar upload; `avatarUrl` is a string only, `schema.prisma:23`; presence exists but is split)	P1 (JWT), P3 (UI). Chat must hand over: friends list + presence.	Overlaps "Allow users to interact"
User Management: "Minor: Implement remote authentication with OAuth 2.0"	Minor, 1	OAuth with Google/GitHub/42	Auth teammate	Not chat	None for chat	None
User Management: "Major: Advanced permissions system"	Major, 2	CRUD users, roles (admin, moderator...), views by role	Chat could own part	Not started (`Role` table unused; no checks)	Stretch: could be shared with auth	Needs auth
User Management: "Major: An organization system"	Major, 2	Create/edit/delete organizations; add and remove users; view; actions (create, read, update)	Chat could own (guilds = organizations)	Started. Guild create and invite-join exist in the API only; no edit, delete, leave or kick (audit P2-13) and no React UI. Whether an evaluator accepts a chat guild as an "organization" is uncertain.	Stretch after P3: add edit/delete guild, leave and kick, plus UI	Needs auth
User Management: "Minor: Implement a complete 2FA" / "Minor: User activity analytics" / "Minor: Game statistics and match history"	Minor, 1 each	See subject	Other	Not chat	Not chat. Game statistics require a game.	Game
AI: "Minor: Voice/speech integration for accessibility or interaction."	Minor, 1	Voice/speech for accessibility or interaction	Chat (borderline)	Not started (voice is a stub; `chat.service.ts:501-519` flips a DB flag)	A WebRTC call is not obviously "speech integration". Safer to present voice under Modules of choice.	None
AI: "Minor: Content moderation AI" / "Minor: Sentiment analysis" etc.	Minor/Major	AI features	Other	Not started	Do not claim	None

Module (exact name)	Type, pts	Subject criteria (short)	Owner	Status now	Gap, blueprint phase	Dependencies
Cybersecurity: "Major: Implement WAF/ModSecurity (hardened) + HashiCorp Vault for secrets"	Major, 2	ModSecurity + Vault	Devops teammate	Not started	Not chat	None
Gaming: "Minor: Advanced chat features (enhances the basic chat from 'User interaction' module)."	Minor, 1	Block users; invite users to play games from chat; game/tournament notifications in chat; access to profiles from chat; chat history persistence; typing indicators and read receipts	Chat	**Started**. History persistence: backend yes. Typing: gateway handles it (`gateway.ts:206`) but no UI sends or shows it. Read receipts: endpoint returns `{ok:true}` and does nothing (`app.controller.ts:185-188`). Block: absent. Profile from chat: no UI. Game items: no game.	P3 plus later work. Only after the basic chat module is truly done.	**Requires** the basic chat module. Two bullets require a game; today no game is known.
Gaming: games, tournaments, customization, spectator, etc.	Major/Minor	Need a game	Other teammates	Not chat	None. Chat only touches them via invites and notifications.	Game
Devops: "Major: Backend as microservices."	Major, 2	"loosely-coupled services with clear interfaces"; "REST APIs or message queues for communication"; "single responsibility"	Shared (counts once for the team); chat provides 2 services	**Near / Started**. chatsystem (REST writes) and gateway (WS) talk through Kafka (a message queue). Weak points: gateway reads Postgres directly, so the DB is shared; Kafka publish failure silently drops delivery (`kafka.service.ts:22-30`); the voice service is a stub; the other teammates' services are unknown	P2, P4 (voice signaling as a real service). Document each service's single responsibility and its interface.	Needs auth as its own service or at least a clear interface to count well
Devops: "Major: Infrastructure for log management using ELK" / "Major: Monitoring system with Prometheus and Grafana" / "Minor: Health check and status page system..."	Major/Minor	See subject	Devops teammate (likely)	Not started	Not chat	None
Data: "Major: Advanced analytics dashboard", "Minor: Data export and import", "Minor: GDPR compliance features"	Major/Minor	See subject	Other	Not started	Do not claim	None
Blockchain modules	Major/Minor	Tournament scores; ICP backend	Other	Not chat	None	Game
Modules of choice: "Major: Implement a custom module that is not listed above." / "Minor: Same as the major module but smaller in scope"	Major 2 / Minor 1	Substantial, with README justification (why, challenges, value, why Major)	Chat	Not started (voice is the candidate)	P4: WebRTC voice with a signaling relay. Claim it as the Minor first; claim it as Major only if the team can defend the extra scope in the README.	Needs the chat running with real users

Notes on the table:

- The subject has no module literally called "voice chat" or "video". The only voice-related entries are "Voice/speech integration" (AI) and the Modules of choice. This is a risk to confirm with the evaluator or staff.
- Docker, HTTPS and the privacy pages are mandatory, so they earn no points.

3. Points ledger (chat-owned work)

Module	Pts	Claimable NOW	After MVP (P0–P5)	Stretch	Counts once for the team? Who demos
Major: Use a framework for both the frontend and backend	2	2 (caveat: Chrome, console)	2		Yes. Whoever knows the stack; the chat owner can demo the Nest + React parts
Minor: Use an ORM for the database	1	1	1		Yes. Chat owner demos the Prisma schema and migrations
Major: Implement real-time features using WebSockets or similar technology	2	0	2		Chat owner demos two browsers live
Major: Allow users to interact with other users	2	0	2		Chat owner demos chat; the auth teammate demos profile data. Agree who shows what
Minor of choice: WebRTC voice channels (justified in README)	1	0	1	up to 2 as Major if defensible	Chat owner
Major: Backend as microservices	2	0	2 (only if the team's services also hold up)		Yes; Tech Lead demos, chat owner explains chatsystem + gateway + Kafka
Minor: Advanced chat features	1	0	0	1 (if no game, ask the staff first)	Chat owner
Major: An organization system	2	0	0	2 (uncertain fit)	Chat owner
Total		**3**	**about 8–10 depending on shared items**	up to +3	

Honest reading of the "after MVP" column: frameworks 2 + ORM 1 + real-time 2 + interact 2 = 7 firm. Add voice 1 = 8. Add microservices 2 = 10 if the whole team's services qualify. The figure of about 7–8 chat-owned in the summary leaves microservices out because it needs the other teammates.

Shared modules count once for the team. If two people claim the same module, the points are not doubled. Decide up front who stands in front of the evaluator for each.

4. Evaluation-day risks (current result against the chat)

Evaluator action	Current result	Why
Spoof another user (`POST /api/rooms/1/messages {"senderId":"3"}` , `PATCH /api/users/1` without credentials)	**FAIL**	Returned 201 and 200 in the audit. Identity is body-supplied (`app.controller.ts:106-116`).
Read someone else's DMs	**FAIL**	Both ids come from the URL (`app.controller.ts:166-183`).
WebSocket IDENTIFY with someone else's id as token	**FAIL**	"token == user id" (`session-loader.service.ts:14-25`).
Two browsers, two users, message appears live	**Partial**. Passes for guild rooms (audit step 7e, Kafka → gateway → WS, using a script, not a browser). Not verified in the UI.	
Two browsers, DM live	**FAIL**	DMs not indexed (`session-loader.service.ts:54-58`). React cannot open the DM screen (`ui.jsx:472`).
Create a guild or room, other user sees it	**FAIL**	No UI; membership frozen at connect.
Edit or delete a message	**FAIL**	No UI; backend events arrive as `MESSAGE_CREATE` .
Reload the page and see history	**Likely pass** for the selected room (`GET /rooms/:id/messages`); not browser-tested. Resync on reconnect is ignored.	
Kill the network, reconnect	**Partial**. A reconnect loop exists but the delay never resets (`gatewayClient.ts:18, :93`) and there is no resync.	
Open the browser console	**Unknown**. Probably warnings or errors.	
Open over HTTPS	**FAIL**	Only `http://` and `ws://` .
Run one command to start everything	**FAIL**	No Dockerfiles; compose has infra only.
Check Privacy Policy and Terms	**FAIL**	None exist.
Check `.env` in git	**FAIL**	Tracked, with a password.
Try SQL injection or XSS in a message	**Likely pass**	Prisma parameterizes; React escapes. Not re-tested.
Send an empty or 5000-char message, bad ids	**FAIL** for bad ids (500s); the 4000-char limit exists.	
Voice call between two users	**FAIL**	Fake button and fake `00:00` timer (`ui.jsx:214-216`).
Explain the architecture and Kafka choice	Depends on you	Be ready to justify Kafka and the two-service split.
"Brief modification" in the code	Depends on you	`chat.service.ts` is 547 lines and `ui.jsx` 539 lines in one file. Know where the send and receive paths are.

5. Recommendations

Claim (priority = points per effort)

1. **Frameworks (Major, 2) and ORM (Minor, 1).** Almost free. You already have them. Only the Chrome and console cleanliness remain.
2. **Real-time WebSockets (Major, 2).** The core path already works. The work is a bounded list: DM indexing, edit/delete events, lazy membership, room filter, reconnect reset, auth on the socket.
3. **Allow users to interact (Major, 2).** Backend mostly exists. You need remove-friend, a profile view, a DM screen and a friends list in React. Agree with the auth teammate who supplies the profile fields.
4. **Voice as Modules of choice (Minor, 1).** Claim it as a Minor first. Write the README justification (why, challenge, value). Mesh WebRTC with signaling through the gateway is a defensible Minor. Only argue Major if you add something clearly harder.
5. **Backend as microservices (Major, 2), as the team's claim.** Your chatsystem + gateway + queue is a good part of it. It only counts if the other services are also separate with clear interfaces. Ask the Tech Lead.

Leave to teammates

- Standard user management, OAuth and 2FA (auth teammate).
- HTTPS proxy, compose single-command, ELK, Prometheus/Grafana, WAF/Vault (devops, likely).
- Games, tournaments, game customization, spectator mode (game owner).
- Privacy Policy and Terms: one team page. Offer a paragraph about chat data.

Do not claim

- Notifications, file upload, search, public API, WCAG, i18n/RTL, additional browsers, PWA, design system, real-time collaboration, analytics, GDPR, AI modules. Each is large for the points and none is near.
- Advanced chat until the basic chat module is truly done and you have asked whether it can be claimed without a game. Treat it as a stretch.
- Organization system, unless time remains after P3. It is 2 points but the "guild = organization" argument is uncertain.

Order of work against the subject

1. P0 first: Dockerfiles in one compose, HTTPS via the team proxy, untrack `.env` / `node_modules` / `dist`, `.env.example`, real `.gitignore`. These are rejection-level.
2. P1: JWT from the auth teammate in REST and WebSocket. Drop client-supplied ids.
3. P2 then P3 for the real-time and interact modules. Test them with two real browsers before saying they are done.
4. P4 voice, only if P0–P3 are green.
5. P5: tests, README sections, Chrome console pass, rehearse the demo of each claimed module.

If the team is short of 14 points by the final week, drop voice before dropping any module that is already near done. A module that fails the demo is worth 0, so better to claim fewer modules that work than more that do not.